



第四章

进程同步和互斥

刘松冉

计算机学院计算机工程系





Review



- 进程控制
 - ✓ 原语
 - ✓ 进程控制原语：创建、撤销、阻塞与唤醒、挂起与激活
 - ✓ UNIX的进程、状态、控制原语（fork）
- 线程
 - ✓ 引进线程的目的
 - ✓ 线程的属性
 - ✓ 线程的状态
 - ✓ 线程与进程的区别
 - ✓ 线程的创建方式：用户级、核心级、轻量进程



第四章 进程同步和互斥

- ✓ 为了描述程序在并发执行时对系统资源的共享，需要一个描述程序执行时动态特征的概念，这就是进程。
- ✓ 在学习了进程的概念、进程描述、进程控制和线程之后，本章将进程间关系-进程互斥和同步。

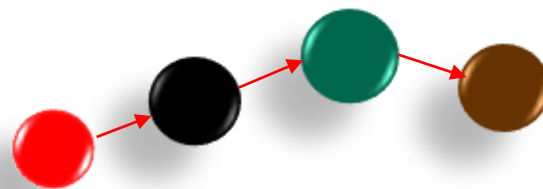
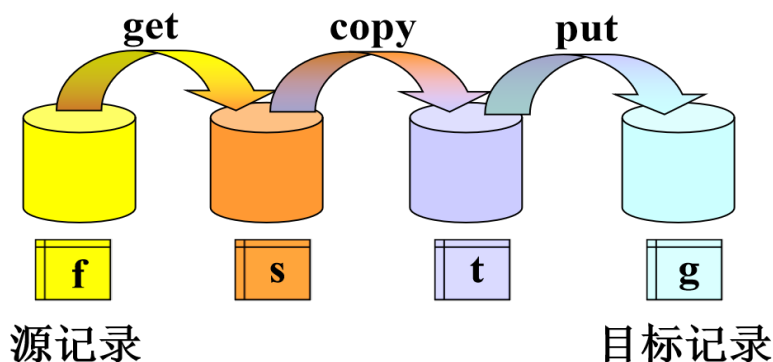
1. 概述
2. 进程的描述
3. 进程控制
4. 线程
5. 进程互斥和同步
6. 进程间通信
7. 死锁问题
8. 进程其他方面的举例



4 进程同步和互斥

本章要解决的问题：

- 多进程（线程）如何实现正确地并发？
- 如何访问共享资源？
- 互斥算法如何实现？
- 多个互相合作的进程如何并发？





4 进程互斥和同步

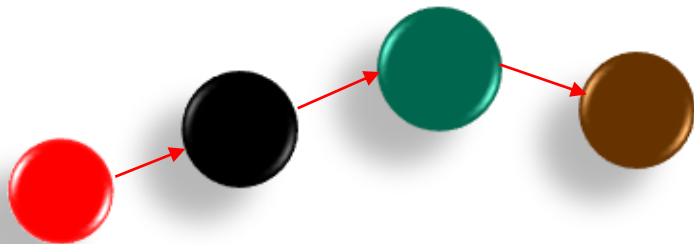


本章内容提要:

1. 进程间关系
2. 互斥算法
3. 信号量及其P、V原语
4. 经典同步问题: 生产者-消费者、读者-写者、哲学家进餐
5. 信号量集
6. 管程
7. Linux的进程同步机制

目标:

- ✓ 学习并掌握主要的概念, 如临界资源、临界区、同步、互斥、信号量等;
- ✓ 了解同步互斥的软、硬件实现方法, 能够分析实现方法存在的问题;
- ✓ 掌握信号量概念及其P、V原语, 及其各种演化版本, 能够利用信号量及其P、V操作正确实现进程控制;
- ✓ 了解经典同步互斥问题, 掌握利用信号量的解决方法, 并能够借鉴其解决方法, 解决其他的问题。
- ✓ **素养:** 通过本章的学习, 体会**遵守规则、团结协作、科学家精神**, 培养**分析问题的能力**。





4 进程互斥和同步

回顾操作系统的特性：

- 并发
- 资源共享
- 异步性（不确定性 或 随机性）
- 虚拟性



4.1 进程间的关系

内容:

- ① 进程（或程序段）的并发执行分析：3个例子；
- ② 进程间的交互：介绍几个重要的概念：
 - 间接制约、直接制约、互斥、同步、临界资源、临界区
 - 实例

目标:

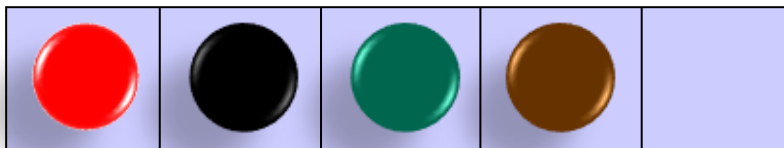
- ① 通过分析进程（两个程序段）的不限制并发执行产生的问题，引出并发执行应该考虑的问题，培养分析问题、提出问题能力；
- ② 通过进程间的两种交互，引出直接制约、间接制约、临界资源、临界区、同步、互斥的概念，能够理解和清晰地描述这些概念；并能给出现实生活中的实例；
- ③ 培养学生的分析问题的能力；培养学生遵章守纪意识、团结协作精神。



4.1 进程间的关系

4.1.1 并发执行举例：

例1：利用堆栈管理一块内存区中各数据块的使用情况。用`getaddr(top)`从栈顶取出相应的内存块的地址。用`reladdr(blk)`将数据（以`blk`为地址）放入堆栈中。





4.1 进程间的关系

4.1.1 并发执行举例：

```
proc getaddr()  
Begin  
  local r;  
  1.1 r ← s[top];  
  1.2 top ← top+1;  
  1.3 return (r);  
End
```

```
Proc reladdr(blk)  
Begin  
  2.1 top ← top-1;  
  2.2 s[top] ← blk;  
End
```

```
Process A  
Begin  
  .....  
  getaddr()  
  .....  
End
```

```
Process B  
Begin  
  .....  
  reladdr(blk)  
  .....  
End
```

序号	可能的执行顺序	结果
1	1.1, 1.2, 1.3, 2.1, 2.2	? 正确
2	2.1, 2.2, 1.1, 1.2, 1.3	? 正确
3	2.1, 1.1, 2.2, 1.2, 1.3	? 错误
4	1.1, 2.1, 2.2, 1.2, 1.3	? 混乱

为什么出错？怎么办？



4.1 进程间的关系

4.1.1 并发执行举例：

- **例2：**一飞机订票系统，两个终端，运行T1、T2进程

T1 :	T2:
...	...
Read(x);	Read(x);
y1:=x;	y2:=x;
if y1>=1 then	if y2>=1 then
y1:=y1-1;	y2:=y2-1;
x:=y1;	x:=y2;
write(x);	write(x);
...	...

设x的当前值为： 100

1. 若执行顺序为：
先T1；后T2；
则结果： $x = 98$
2. 若执行顺序为：
T1: Read(x); y1:=x;
T2: Read(x); y2:=x;
T1: if y1>=1 then y1:=y1-1;
T2: if y2>=1 then y2:=y2-1;
T1: write(x);
T2: write(x);
则结果 $x = 99$



- **分析：**产生错误的原因是什么？
- **原因：**不加控制地访问共享变量x



4.1 进程间的关系

4.1.1 并发执行举例：

例3：与进程的执行顺序有关的错误

复制一个记录

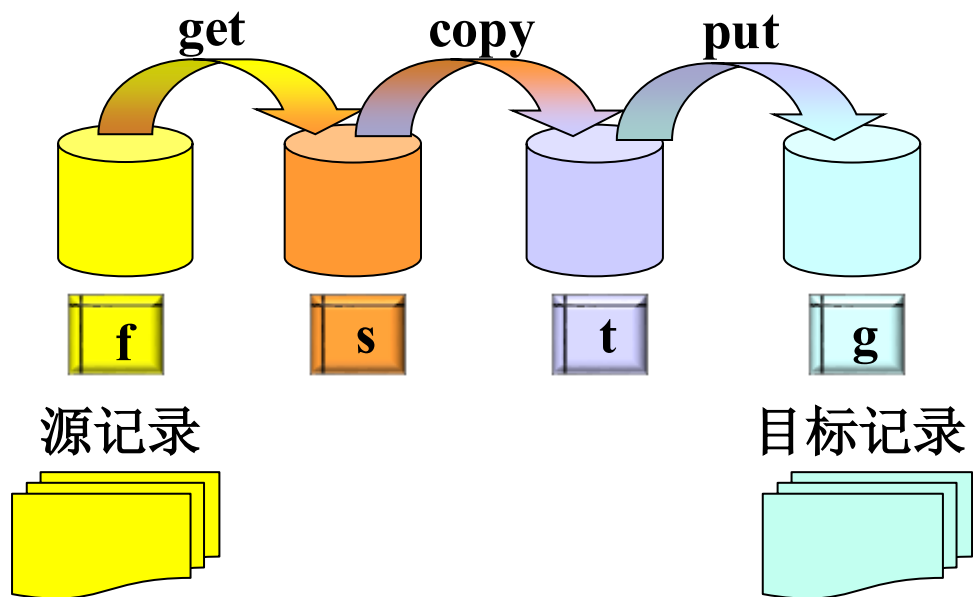
Cobegin

get;

copy;

put;

Coend





4.1 进程间的关系

4.1.1 并发执行举例：

例3：与进程的执行顺序有关的错误

序号	初始状态 及执行顺序	源记录的 剩余部分	目标记录的 当前值	结果正确性
0	初始状态	^f 3,4,...,m ^s 2 ^t 2	^g (1,2)	
1	g,c,p	4,5,...,m ³ ³	(1,2,3)	√
2	g,p,c	4,5,...,m ³ ³	(1,2,2)	X

Diagram illustrating the execution sequence and state changes for three processes (f, s, t) and a target record (g). Red arrows show the flow of updates from the source records to the target record.

注：从1到6的各个执行，是在0（初始状态）的基础上进行的



4.1 进程间的关系

4.1.1 并发执行举例：

例3：与进程的执行顺序有关的错误

序号	初始状态 及执行顺序	源记录的 剩余部分			目标记录 的当前值	结果正确性
		f	s	t	g	
0	初始状态	3,4,...,m	2	2	(1,2)	
1	g,c,p	4,5,...,m	3	3	(1,2,3)	✓
2	g,p,c	4,5,...,m	3	3	(1,2,2)	X
3	c,g,p	4,5,...,m	3	2	(1,2,2)	X
4	c,p,g	4,5,...,m	3	2	(1,2,2)	X
5	p,c,g	4,5,...,m	3	2	(1,2,2)	X
6	p,g,c	4,5,...,m	3	3	(1,2,2)	X

注：从1到6的各个执行，是在0（初始状态）的基础上进行的

错误的原因：是get, copy, put 过程没有按照应该的顺序执行

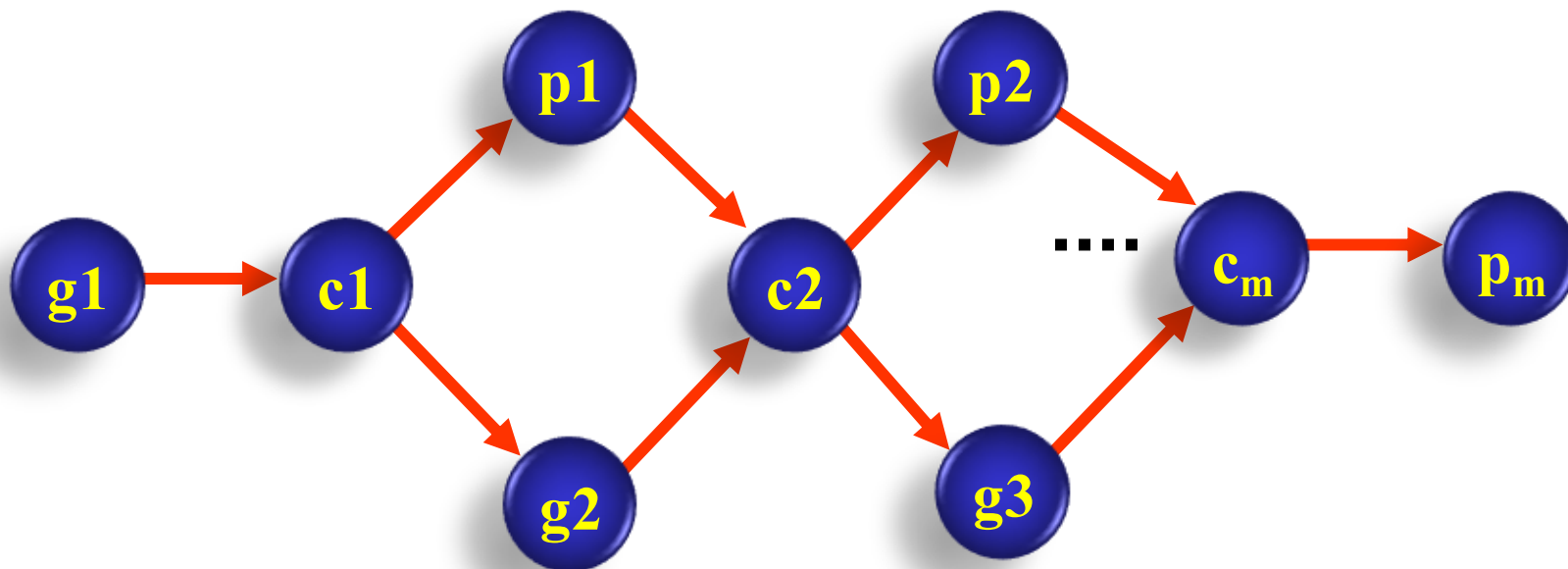


4.1 进程间的关系

4.1.1 并发执行举例：

例3： 与进程的执行顺序有关的错误

● 正确执行顺序



注：上图中g、c、p分别代表get、copy和put进程，1、2、3、...代表第几条记录



4.1 进程间的关系

4.1.2 进程间的交互

● **进程的交互关系：** 可以按照**相互感知的程度**来分类

- **互斥：** 指多个进程不能同时使用同一个资源；
- **同步：** 指多个并发进程相互合作，完成任务
- **死锁：** 指多个进程互不相让，都得不到足够的资源；
- **饥饿：** 指一个进程一直得不到资源（其他进程可能轮流占用资源）。



4.1 进程间的关系

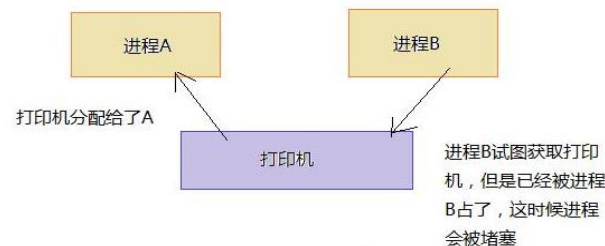
4.1.2 进程间的交互

在多道程序环境中，对于同一个系统中的多个进程，由于它们**共享系统中的资源**，或者为完成某一任务而**相互合作**，使得它们之间存在两种**制约关系**：**间接制约**和**直接制约**

间接制约：

- 进行竞争——**独占**分配到的部分或全部共享资源，“**互斥**”。
- 进程间要通过某种**中介**发生联系，是**无意识**安排的，可发生在相关进程之间，也可发生在无关进程之间。如：例1、例2。
- **定义**：因为**共享公有资源**而造成的对并发进程的**执行速度的制约**，称为**间接制约**。

➤ **例子：??** 间接制约：





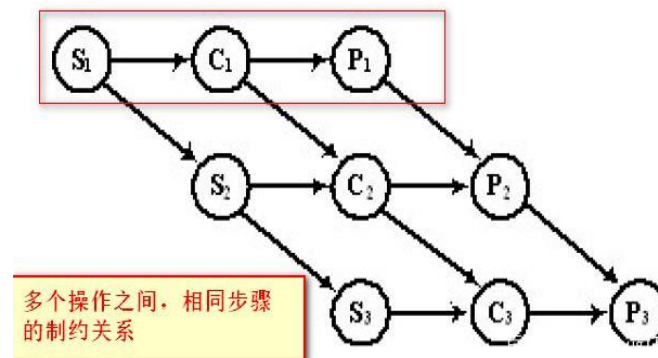
4.1 进程间的关系

4.1.2 进程间的交互：

在多道程序环境中，对于同一个系统中的多个进程，由于它们共享系统中的资源，或者为完成某一任务而相互合作，使得它们之间存在两种制约关系：间接制约vs直接制约

直接制约：

- ✓ 进行协作——等待来自其他进程的信息，“同步”。
- ✓ 进程间的相互联系是**有意识**的安排的，直接制约只发生在相关进程间。
- ✓ **定义：**一组异步环境下的并发进程，各自的执行结果为对方的执行条件，从而限制各进程的执行速度的现象，称为进程间的**直接制约**。如：例3。
- ✓ 举例：??





4.1 进程间的关系

4.1.2 进程间的交互：

简介制约与直接制约的对比

维度	直接制约	间接制约
定义	异步并发进程间因逻辑协作关系需按特定顺序执行	异步并发进程间因共享独占资源而造成执行速度的制约
产生原因	进程间存在功能依赖（如数据传递），直接的	进程竞争同一不可共享资源（如打印机），间接的
典型场景	生产者-消费者问题、读写协作	多进程访问临界资源（如内存、设备、共享变量）
资源类型	共享缓冲区、信号量、管道等通信资源	物理设备、数据文件等独占资源
相互联系	有意安排	无意安排



4.1 进程间的关系

4.1.2 进程间的交互

● 临界资源 (critical resource)

- 系统中某些资源一次只允许一个进程使用，这样的资源为**临界资源**或互斥资源或共享变量。如，例1中的栈，例2中的变量x，打印机。

● 临界区(Critical region):

- 不允许多个并发程序**交叉执行的一段程序**称为临界区。或，
- 访问公有数据（临界资源）的那段程序称为**临界区**。

● 进程的互斥（间接作用） mutual exclusion

- **互斥**：不允许两个及以上的共享公有资源的并发进程同时进入临界区的规则，称为互斥。
- 例：食堂的读卡机的访问、any more?



4.1 进程间的关系

4.1.2 进程间的交互

🌐 进程的同步（直接作用） **synchronism**

- 系统中一些进程需要**相互合作**，**共同**完成一项任务；
- 具体说，一个进程运行到某一点时要求另一伙伴进程为它提供消息，在未获得消息之前，该进程处于等待状态，获得消息后被唤醒进入就绪态。
- 举例：接力赛、🏊‍♂️我！
- **同步定义**：异步环境下的一组并发进程因**直接制约**而互相发送消息而进行互相合作、互相等待，使得各进程按照一定的速度执行的过程称为进程间的**同步**。

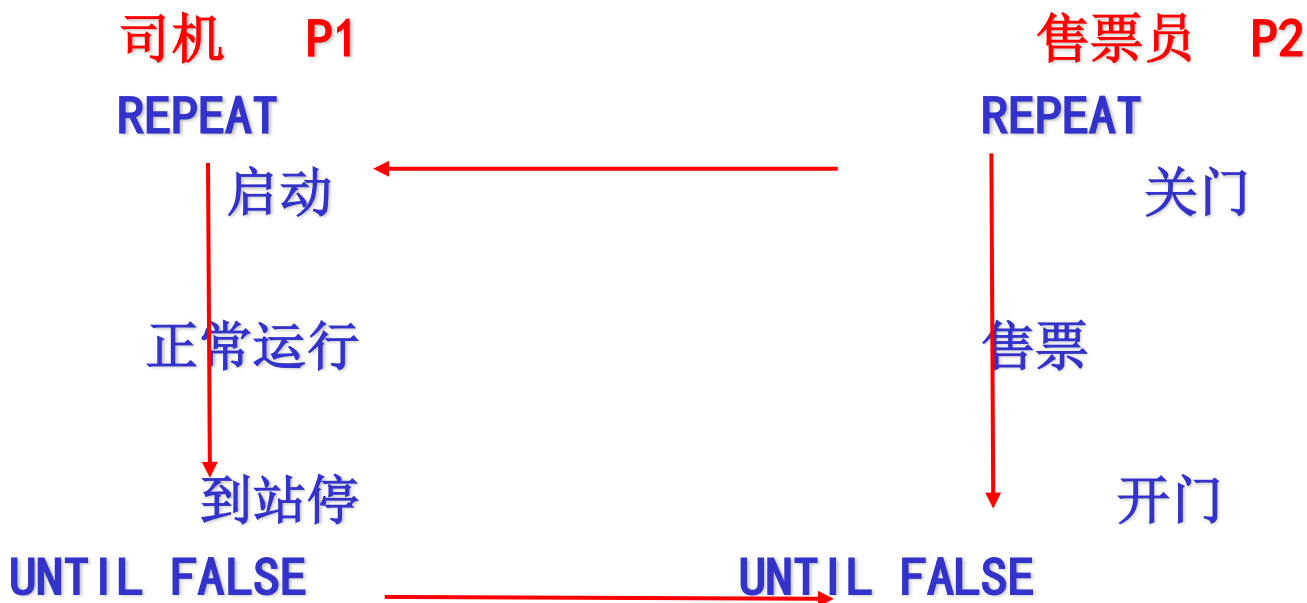




4.1 进程间的关系

4.1.2 进程间的交互

例如：公共汽车的司机和售票员





4.1 进程间的关系

小结:

1. 进程间交互关系

- 互斥、同步、死锁、饥饿

2. 进程间的制约关系：直接制约、间接制约

3. 临界资源、临界区

4. 互斥、同步

5. 启发:

- 现实生活中，有些时候需要遵守“互斥”原则，例如“电话间”的使用，很多时候需要利用“同步”协议，团结协作；再例如软件系统的开发，就需要遵守“合作”，接口的定义，进度的协调等。

- 如何实现互斥，4.2节介绍



4.2 互斥算法

实现对共享变量的正确访问

- ① 临界区的访问过程
- ② 使用临界区应遵循的准则
- ③ 进程互斥的软件方法
- ④ 进程互斥的硬件方法



4.2 互斥算法

4.2.1 临界区的访问过程

entry section

进入区：在进入临界区之前，检查可否进入临界区的一段代码。如果可以进入临界区，通常设置相应“正在访问临界区”标志。

critical section

临界区：进程中访问临界资源的一段代码

exit section

退出区：用于将“正在访问临界区”标志清除

remainder section

剩余区：代码中的其余部分



4.2 互斥算法

4.2.2 使用临界区应遵循的准则

- ① **有空让进**：当无进程在互斥区时，任何有权使用互斥区的进程可进入
- ② **无空等待**：不允许两个以上的进程同时进入互斥区
- ③ **有限等待**：任何进入互斥区的要求应在有限的时间内得到满足
- ④ **让权等待**：处于等待状态的进程应放弃占用CPU
- ⑤ **多中择一**：当没有进程在临界区，而同时有多个进程要求进入临界区，只能让其中之一进入临界区，其他进程必须等待
- ⑥ **平等竞争**：任何进程无权停止其它进程的运行，进程之间相对运行速度无硬性规定



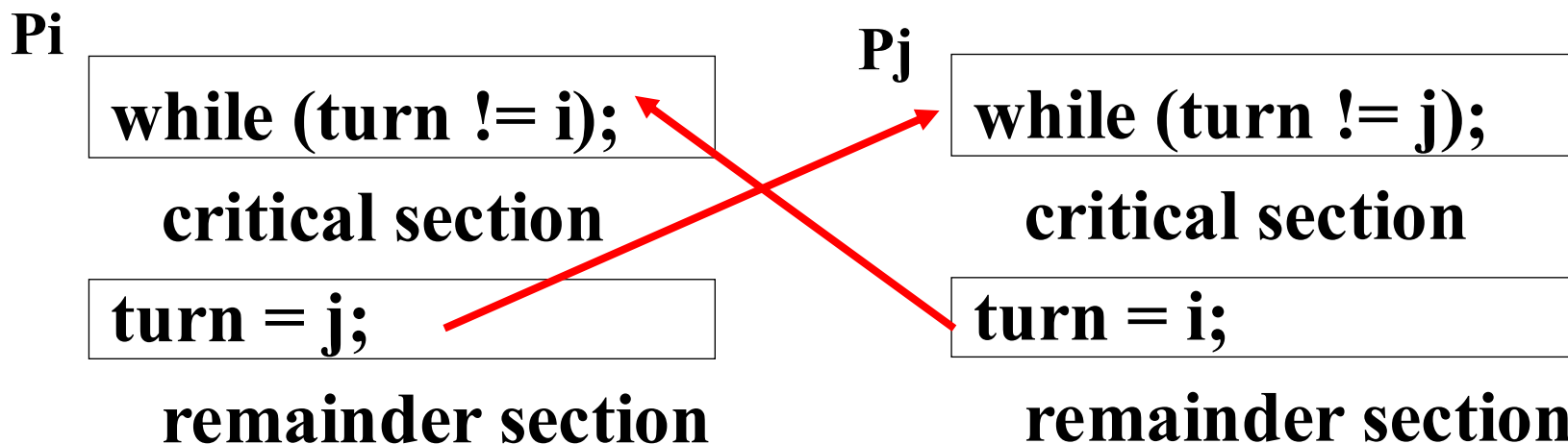
4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法1：单标志

- 设立一个公用整型变量 **turn**：描述允许进入临界区的进程标识

有两个进程 P_i 、 P_j



可以实现互斥进入临界区吗？存在什么问题？想一想

在 P_i 出让临界区之后， P_j 使用临界区之前， P_i 不可能再次使用临界区



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法1：单标志

➤ 设立一个公用整型变量 **turn**：描述允许进入临界区的进程标识

- ✓ 在进入区循环检查是否允许本进程进入：turn为 i 时，进程 P_i 可进入
- ✓ 在退出区修改允许进入进程标识：进程 P_i 退出时，改 turn 为进程 P_j 的标识 j

➤ 缺点：

- ✓ **强制轮流**进入临界区，没有考虑进程的实际需要。容易造成**资源利用不充分**：在 P_i 出让临界区之后， P_j 使用临界区之前， P_i 不可能再次使用临界区。



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法1：另一种单标志

P_i:

```
While flag < > 0;
```

```
    flag = 1;
```

```
    critical section;
```

```
    flag = 0;
```

```
    remainder section;
```

P_j:

```
While flag < > 0;
```

```
    flag = 1;
```

```
    critical section;
```

```
    flag = 0;
```

```
    remainder section;
```

? 可以吗?



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法2：双标志、先检查

P_i:

```
while (flag[j]);    <a>  
flag[i] = TRUE;    <b>
```

critical section

```
flag[i] = FALSE;
```

remainder section

P_j:

```
while (flag[i]);    <a>  
flag[j] = TRUE;    <b>
```

critical section

```
flag[j] = FALSE;
```

remainder section

能互斥吗？存在什么现象？

现象：P_i 和 P_j可能同时进入临界区，why？



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法2：双标志、先检查

- 设立一个标志数组flag[]：描述进程是否在临界区，初值均为FALSE。
- 先检查，后修改：在进入区检查另一个进程是否在临界区，不在时修改本进程在临界区的标志
- 在退出区修改本进程在临界区的标志



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法2：双标志、先检查

- 优点：不用交替进入，可连续使用；
- 缺点： P_i 和 P_j 可能同时进入临界区。
 - 在 P_i 和 P_j 都不在临界区时，假设按下面序列执行时，会同时进入：“ $P_i < a >; P_j < a >; P_i < b >; P_j < b >$ ”。即在检查对方 flag 之后和切换自己 flag 之前有一段时间，结果都检查通过。
 - 这里的问题出在检查和修改操作不能连续进行。



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法3：双标志、后检查

P_i:

flag[i] = TRUE; <a>

while (flag[j]);

critical section

flag[i] = FALSE;

remainder section

P_j:

flag[j] = TRUE; <a>

While (flag[i]);

critical section

flag[j] = FALSE;

remainder section

存在问题： P_i和P_j可能都进入不了临界区



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法3：双标志、后检查

- 类似于算法2，与互斥算法2的区别在于先修改后检查。可防止两个进程同时进入临界区。

- 缺点： P_i 和 P_j 可能都进入不了临界区。

- ✓ 例如：在 P_i 和 P_j 都不在临界区时，假设按下面序列执行时： $P_i<a> P_j<a> P_i P_j$ ，则会都进不了临界区。即在切换自己flag之后和检查对方flag之前有一段时间，结果都切换flag，都检查不通过。



4.2 互斥算法

4.2.3 进程互斥的软件方法

🌐 **算法4 (Peterson's Algorithm):** 先修改、后检查、后修改者等待

P_i

```
flag[i] = TRUE; turn = j;  
while (flag[j] && turn == j);
```

critical section

```
flag[i] = FALSE;
```

remainder section

P_j

```
flag[j] = TRUE; turn = i;  
while (flag[i] && turn == i);
```

critical section

```
flag[j] = FALSE;
```

remainder section

分析算法4，能否出现算法3和算法2中的问题？ ?



4.2 互斥算法

4.2.3 进程互斥的软件方法

● 算法4 (Peterson's Algorithm): 先修改、后检查、后修改者等待

- 结合算法1和算法3，是正确的算法
- $turn=j$ ；描述可进入的进程（同时修改标志时）
- 在进入区先修改后检查，并检查并发修改的先后：
 - 检查对方 $flag$ ，如果不在临界区则自己进入——空闲则入；
 - 否则再检查 $turn$ ：保存的是较晚的一次赋值，则较晚的进程等待，较早的进程进入——先到先入，后到等待。



1. 临界资源

✓ OS的特性之一“资源共享性”，因资源有限，导致竞争和共享，不允许并发进程同时访问临界资源

2. 临界区

3. 进程间相互合作-->直接制约 --> 同步

4. 共享公有资源-->间接制约 --> 互斥

5. 互斥的软件实现方法



4.2 互斥算法

4.2.4 进程互斥的硬件方法

- 完全利用软件方法，**有很大局限性**（如不适于多进程），现在已很少采用。
- 可以利用某些硬件指令——其读写操作由一条指令完成，因而保证读操作与写操作不被打断。



4.2 互斥算法

4.2.4 进程互斥的硬件方法

(1) Test-and-Set指令-- TS 指令

该指令读出标志后设置为TRUE:

```
boolean TS(boolean *lock) {  
    boolean old;  
    old = *lock; *lock = TRUE;  
    return old;  
}
```

- 为每种临界资源，设置一个锁变量lock;
- lock表示资源的两种状态：TRUE表示正被占用，FALSE表示空闲。

Test-and-Set指令,可保证在一个指令周期内对一个存储单元的读和写。



4.2 互斥算法

4.2.4 进程互斥的硬件方法

(1) Test-and-Set指令

● 互斥算法 (TS 指令)

```
boolean TS (boolean *lock)
{
    boolean old;
    old = *lock;
    *lock = TRUE;
    return old;
}
```

while TS(&lock);

critical section

lock = FALSE;

remainder section

测试并加锁:
lock (int *lock)
{
 while TS(&lock);
}

解锁:
unlock (int *lock)
{
 ***lock = FALSE;**
}

lock(&lock);

critical section

unlock(&lock);

remainder section

问题： 能实现互斥吗？



4.2 互斥算法

4.2.4 进程互斥的硬件方法

(1) Test-and-Set指令

● 互斥算法 (TS 指令)

- ✓ 利用TS实现进程互斥：每个**临界资源**设置一个**公共布尔变量lock**，初值为FALSE；
- ✓ 在**进入区**利用TS进行检查：有进程在临界区时，重复检查；直到其它进程退出时，检查通过。



4.2 互斥算法

4.2.4 进程互斥的硬件方法

(2) Swap指令(或Exchange指令, XCHG)

- Swap指令（或Exchange指令），可保证在一个指令周期内交换一个寄存器和一个存储单元内容，其工作原理如下：

交换两个字（字节）的内容

```
void SWAP(int *a, int *b)
{
    int temp;
    temp = *a; *a = *b; *b = temp;
}
```



4.2 互斥算法

4.2.4 进程互斥的硬件方法

(2) Swap指令（或Exchange指令）

● 互斥算法（Swap 指令）

利用Swap实现进程互斥：每个临界资源设置一个公共布尔变量lock，初值为FALSE。每个进程设置一个私有布尔变量key

```
key = TRUE;
do {
    SWAP(&lock, &key);
} while (key);
```

critical section

```
lock = FALSE;
```

remainder section

```
void SWAP (int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```



4.2 互斥算法

4.2.4 进程互斥的硬件方法

(3) 开关中断指令

● 互斥算法（开关中断指令）

✓ 进入临界区前：

执行“关中断”指令

✓ 离开临界区后：

执行“开中断”指令

关中断；

critical section

开中断；

remainder section

● 特点：

- 代价高，CPU只能交替执行；
- 不能用于多处理机结构，因为关中断只能屏蔽本机的中断。



4.2 互斥算法

4.2.4 进程互斥的硬件方法

● 硬件方法的优点

- 适用于任意数目的进程，在单处理器或多处理器上（除中断方法）；
- 简单，容易验证其正确性；
- 可以支持进程内存在多个临界区，只需为每个临界区设立一个布尔变量。



4.2 互斥算法

4.2.4 进程互斥的硬件方法

● 硬件方法的缺点

- ① 等待要耗费CPU时间，不能实现“让权等待”；
- ② 可能“饥饿”（不公平）：从等待进程中随机选择一个进入临界区，有的进程可能一直选不上；
- ③ 可能死锁：进程P1执行TS或Exchange指令并进入临界区，然后P1被P2中断并把CPU给具有更高优先级的P2，若P2试图使用与P1相同的资源，由于互斥机制，它被拒绝访问，因此进入忙等待循环，又因P2优先级高于P1，所以P1总得不到调度执行。

● 举例：学生借教室、自己反复地查看教室的状态；

● 解决办法：如果设立了教师管理员，管理员会通知他，当可用时通知最先等待的学生



4.2 互斥算法-小结

- ① 临界区的访问过程
 - 进入区、临界区、退出区
- ② 使用临界区应遵循的准则
 - 有空让进、无空等待、有限等待、让权等待、多中择一、平等竞争
- ③ 进程互斥的软件方法【了解】
 - 单标志、双标志先检查、双标志后检查
 - **Peterson's Algorithm):** 先修改、后检查、后修改者等待
- ④ 进程互斥的硬件方法
 - TS指令、互换指令、关中断



4.3 信号量(semaphore)

解决互斥算法的出发点:

- 前面的互斥算法都存在问题，它们是平等进程间的一种协商/竞争机制；
- 不公平，可能产生饥饿现象
- 需要一个地位高于进程的管理者来解决公有资源的使用问题；
- OS可从进程管理者的角度来处理互斥的问题，信号量就是OS提供的管理公有资源的有效手段
- 举例：教室使用
 - 管理者、使用者
 - 资源不可用则睡眠等待；可用则管理者唤醒等待者



4.3 信号量

为什么引入信号量？

- 用锁机制方便的解决了临界区的互斥访问问题了，但是锁机制似乎并不能适用于更一般的情况。
- 例如：
 - OS某些资源是有多个的，可以让多个进程共同访问，只有当访问的进程多于资源数量的时候才对进程访问加以限制。
 - 然而，使用锁机制的结果是一次只能有一个进程访问资源，这无疑是对资源的一种浪费。
- 锁机制只能解决进程间的互斥访问问题，并不能推广到进程间的同步问题。



4.3 信号量

信号量(semaphore['seməfɔ:(r)])

信号量是一种卓有成效的进程同步工具，在长期且广泛的应用中，信号量机制得到了很大的发展，从整型信号量，经记录型信号量、AND型信号量、到信号量集

- ① 信号量定义
- ② 信号量的P、V原语
- ③ 信号量的实现
- ④ 信号量集



4.3 信号量

4.3.1 信号量和P、V原语

(1) 信号量概念

➤ 1965年，由荷兰学者Dijkstra提出（所以P、V分别是荷兰语的test(proberen)和increment(verhogen)），是一种卓有成效的进程同步机制。

➤ 整型信号量

定义：早期Dijkstra把整型信号量定义为一个用于表示资源数目的整型量S。除初始化之外，只能通过两个标准的原子操作，即P操作和V操作，来访问。

```
P(S) {  
    while(S<=0) ;  
    S--;  
}
```

```
V(S) {  
    S++;  
}
```



4.3 信号量

4.3.1 信号量和P、V原语

(1) 信号量概念

记录型信号量：semaphore是一个数据结构，定义如下：

```
typedef struct {
```

```
    int count;           // 整数值, 资源数量
```

```
    Pointer_PCB queue;   // 进程等待队列, 其中是阻塞在该信号量  
                          // 上的各个进程的內部标识(即PCB结构)  
} Semaphore;
```

```
Semaphore s;
```



4.3 信号量

4.3.1 信号量和P、V原语

(1) 信号量概念

- 信号量只能通过**初始化**和**两个标准的原语**来访问——作为OS核心代码执行，**不受进程调度的打断**
- **初始化**指定一个非负整数值，表示**空闲资源总数**（又称为“资源信号量”）：
 - 若为非负值：表示**当前的空闲资源数**，
 - 若为负值：其绝对值表示**当前等待临界区的进程数**
- **二值信号量(binary semaphore)**：信号量只取0或1值



4.3 信号量

4.3.1 信号量和P、V原语

(2) 信号量的P原语--P(s) 或wait(s)

- 功能：在互斥问题中，申请使用临界资源时调用它

P(Semaphore s) {

--s.count; //表示申请一个资源;

if (s.count < 0) { //表示没有空闲资源;

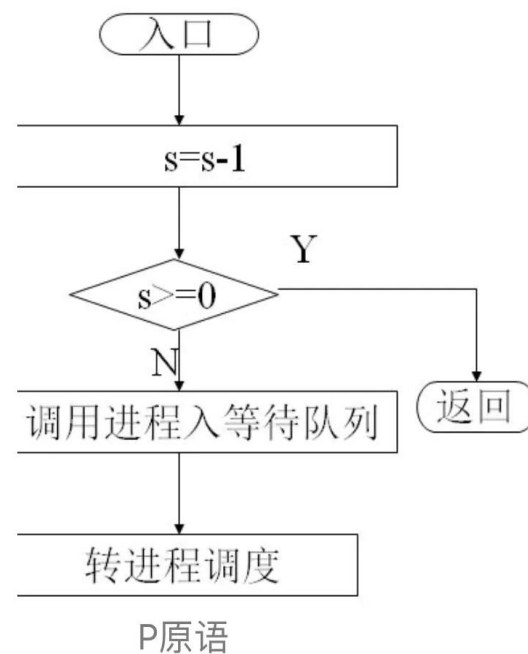
调用者进程进入等待队列s.queue;

阻塞调用者进程;

scheduler();

}

}



- 使用时机：进程进入临界区之前，申请资源



4.3 信号量



4.3.1 信号量和P、V原语

(3) 信号量的V原语—V(s) 或 signal(s)

- 功能：V原语通常释放资源，唤醒进程等待队列中的头一个进程

```
V(Semaphore s) {  
    ++s.count;           //表示释放一个资源;  
    if (s.count <= 0) { //表示有进程处于阻塞状态;  
        从等待队列s.queue中取出一个进程p;  
        进程p进入就绪队列;  
    }  
}
```

- 使用时机：进程退出临界区之后，回收资源



4.3 信号量

4.3.1 信号量和P、V原语

(4) 利用信号量实现互斥

- 为临界资源设置一个互斥信号量mutex (Mutual Exclusion), 其初值为1;
- 在每个进程中将临界区代码置于P(mutex)和V(mutex)原语之间;
- 必须成对使用P和V原语: 遗漏P原语则不能保证互斥访问, 遗漏V原语则不能在使用临界资源之后将其释放 (给其他等待的进程);
- P、V原语不能次序错误、重复或遗漏: 通常是: “申请->使用->释放”

P(mutex);

critical section

V(mutex);

remainder section

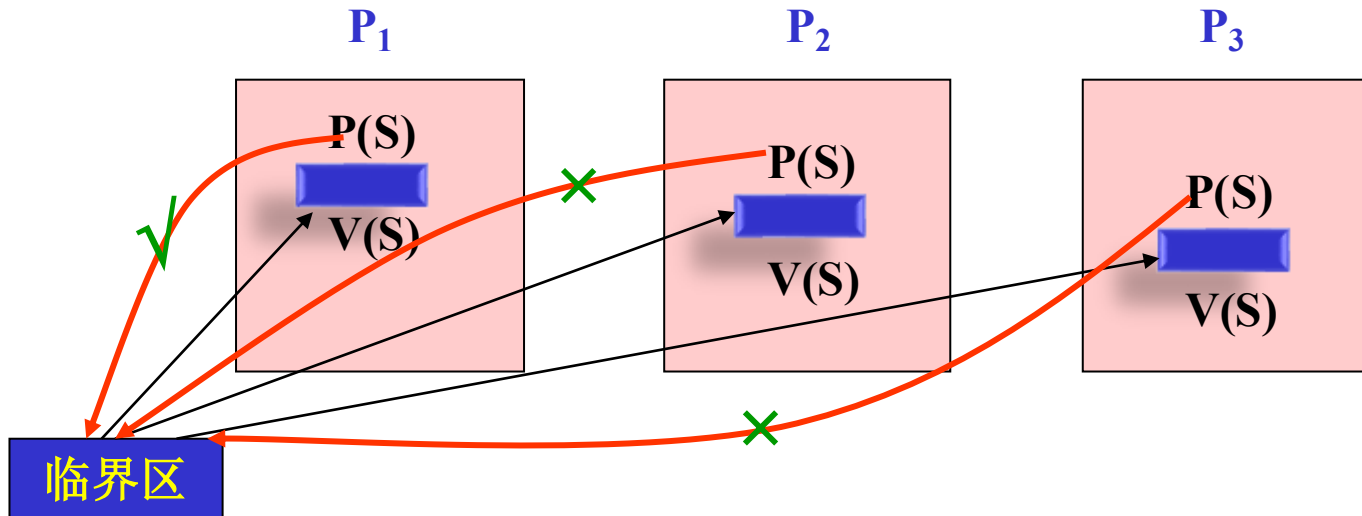


4.3 信号量

4.3.1 信号量和P、V原语

(4) 利用信号量实现互斥

● 图示：用P、V操作解决进程间互斥问题





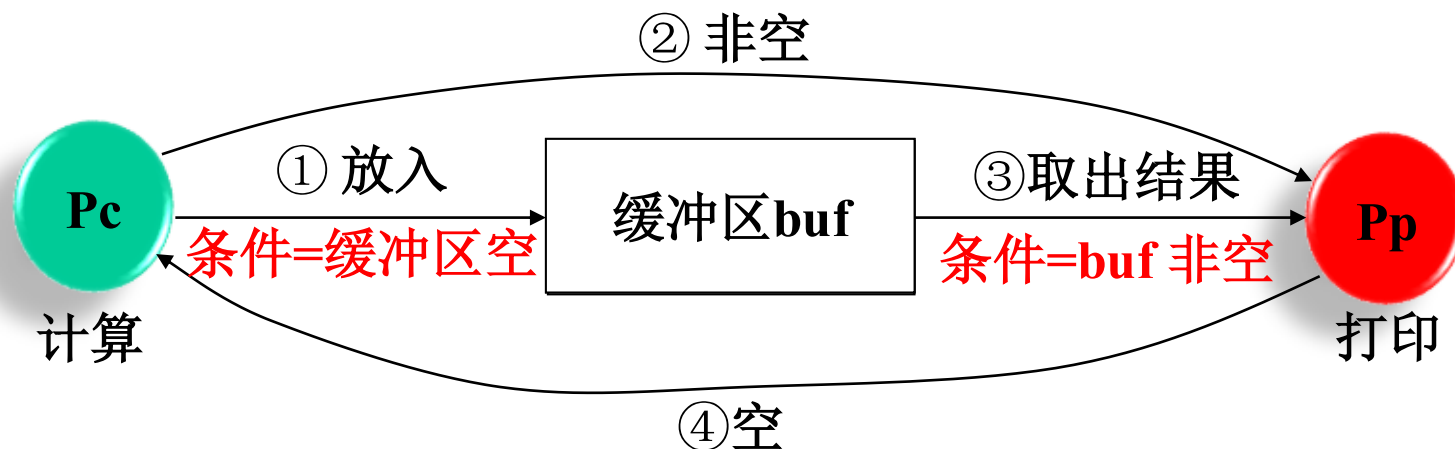
4.3 信号量



4.3.1 信号量和P、V原语

(5) 利用信号量实现进程同步

● 问题1：进程Pc 和 Pp 合作完成计算和打印任务



问题： Pc如何通知Pp缓冲区非空？ Pp如何通知Pc已空？



4.3 信号量

4.3.1 信号量和P、V原语

(5) 利用信号量实现进程同步

● 问题1: 进程Pc 和 Pp 合作完成计算和打印任务

解: 设信号量: bufempty: 表示buf 空, =1
buffull: 表示buf满, =0

Pc:

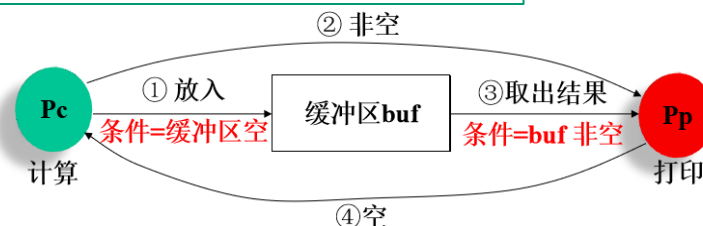
```
A: P(bufempty);  
   计算;  
   计算结果放入buf;  
   V( buffull);  
   Goto A;
```

Pp:

```
B: P(buffull);  
   取出buf中的数据;  
   置空标记, 打印;  
   V(bufempty);  
   Goto B;
```

讨论:

- Pc、Pp如何实现图中的各个步骤?
- 这两个进程是否可以优化?





4.3 信号量

4.3.1 信号量和P、V原语

(5) 利用信号量实现进程同步

- 进程Pc 和 Pp 合作完成计算和打印任务

思考：若Pc和Pp如下

Pc:

Repeat

 计算;

 buf ← 计算结果

 V(buffull);

 P(bufempty)

Until false

Pp:

Repeat

 P(buffull);

 取出buf中的数据

 置空标记, 打印

 V(bufempty);

Until false

则: bufempty初值为? buffull初值为?



Review1

1. 使用硬件进行进程间的互斥:
✓ TS指令、交换指令swap、
关中断
2. 信号量semaphore
3. 信号量实现进程同步和互斥

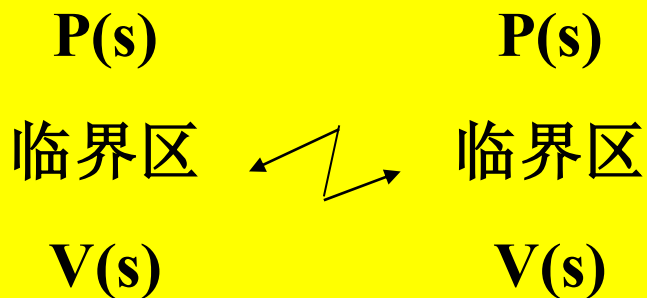
entry section

critical section

exit section

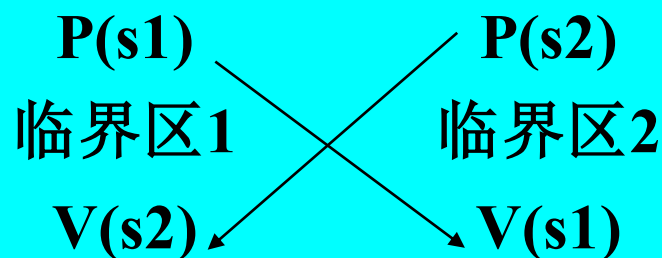
remainder section

S初始值为1



互斥

S1初始值n, s2初始值0

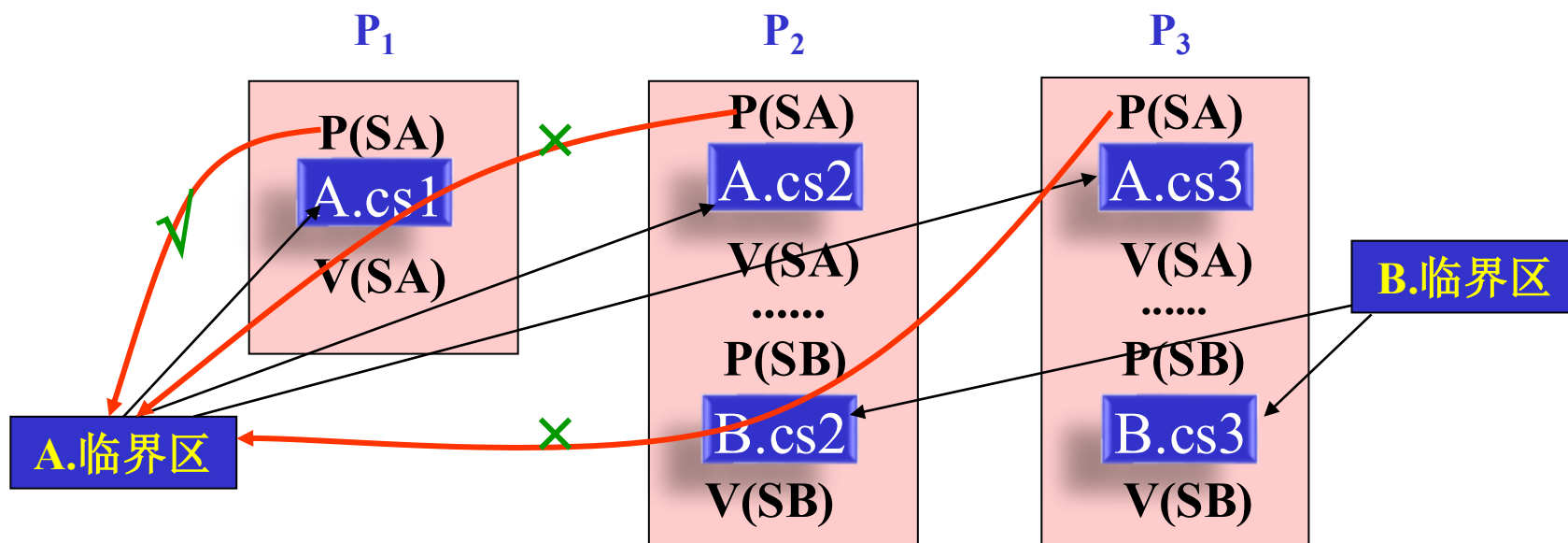


同步



Review2

- **临界区**：是访问临界资源的程序段
- 一种临界资源对应一组访问该种资源的临界区，它们之间要互斥
- 两种不同临界资源的临界区间，需要互斥？
 - A.cs1、A.cs2、A.cs3之间互斥； B.cs2、B.cs3互斥
 - **A.cs3与B.cs3要互斥吗？**





4.3 信号量



4.3.1 信号量和P、V原语

(5) 利用信号量实现进程同步

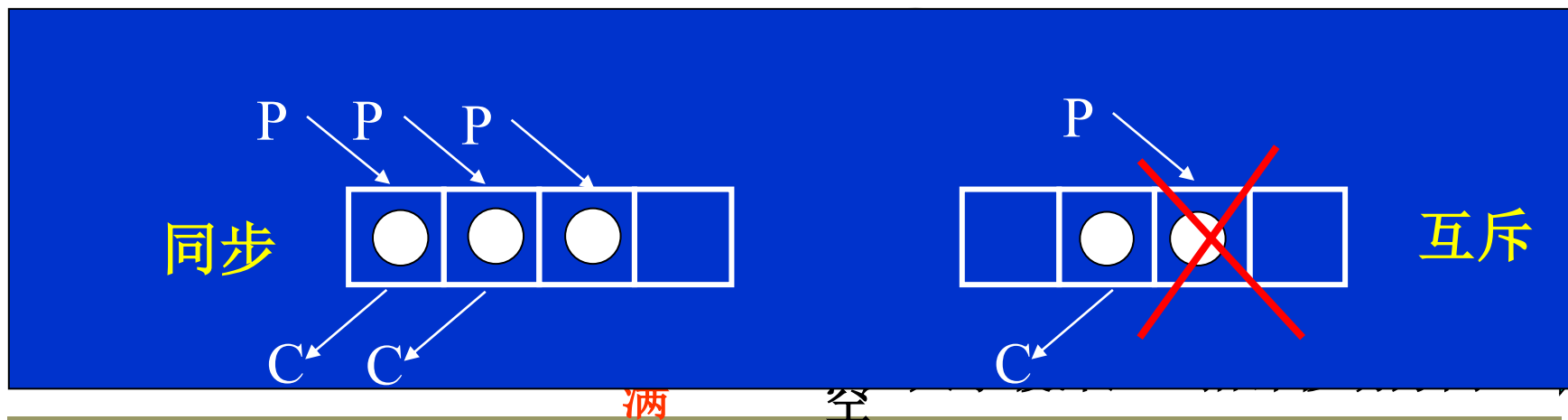
● 生产者/消费者问题 (the producer/consumer problem)

➤ **问题描述:** 若干进程通过有限的共享缓冲区交换数据。其中, "生产者"进程不断写入, 而"消费者"进程不断读出; 共享缓冲区共有N个空间; 任何时刻只能有一个进程可对共享缓冲区进行操作。

➤ **分析:** 生产者和消费者存在什么关系? 需要互斥?

● **同步:** 缓冲区不满, 生产者才能写入; 缓冲区不空, 消费者才能读出

● **互斥:** 多个生产者、消费者进程并发访问缓冲区, 对它的访问要互斥





4.3 信号量

设哪些信号量呢？

- full是“满”数目，初值为多少？ 0
- empty是“空”数目，初值为多少？ N
- 实际上，full和empty是同一个含义： $\text{full} + \text{empty} = N$ ？
- mutex用于访问缓冲区时的互斥，初值是多少？ 1

Producer:

1. 生产一个产品p;
2. P(1); //进入区
3. P(mutex);
4. one unit $\rightarrow$ buffer;
5. V(mutex); //退出区
6. V(2);

Consumer:

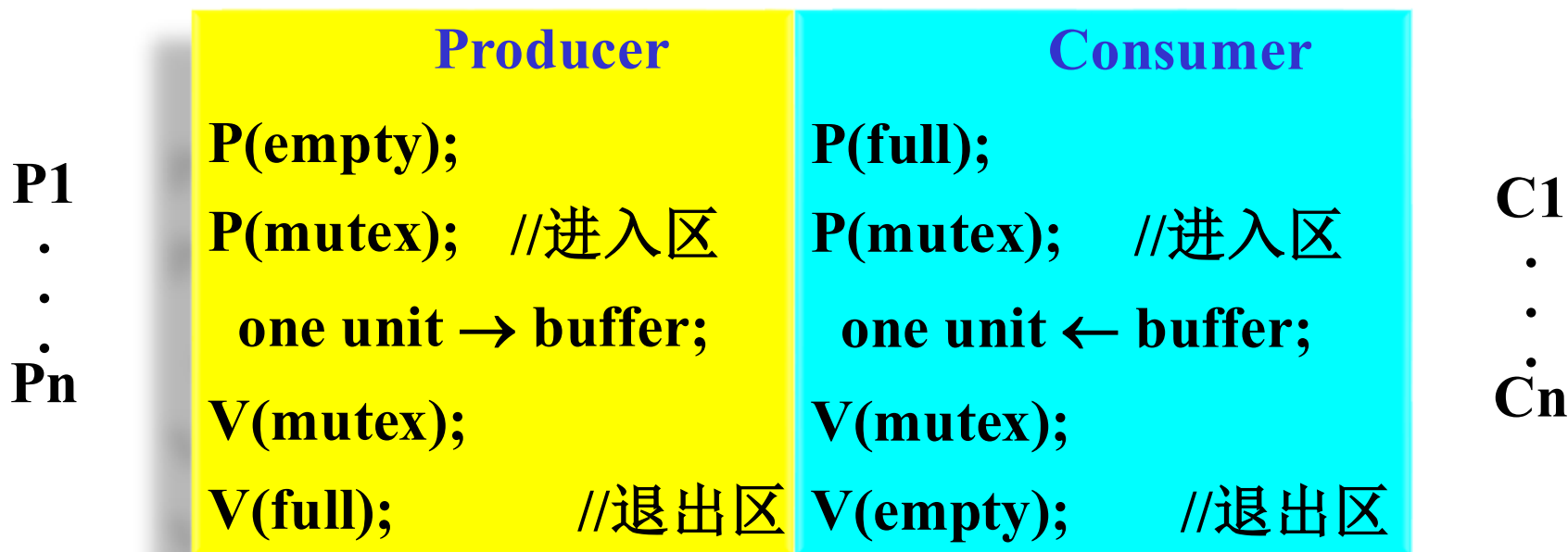
1. P(3); //进入区
2. P(mutex);
3. one unit $\rightarrow$ buffer;
4. V(mutex); //退出区
5. V(4);
6. 消费buffer中的产品;

A. empty B. full

➤ 每个进程中各个P操作的次序是重要的：先检查资源数目，再检查是否互斥——否则可能死锁(为什么?)



4.3 信号量



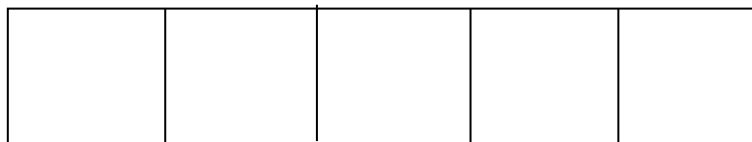
变量:

Empty=5

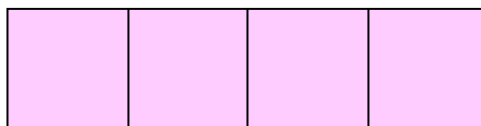
Full=0

Mutex=1

缓冲区



Empty
等待队列





4.3 信号量

P1 
.
.
.
Pn

Producer	Consumer
P(empty);	P(full);
P(mutex); //进入区	P(mutex); //进入区
one unit → buffer;	one unit ← buffer;
V(mutex);	V(mutex);
V(full); //退出区	V(empty); //退出区

C1
.
.
.
Cn

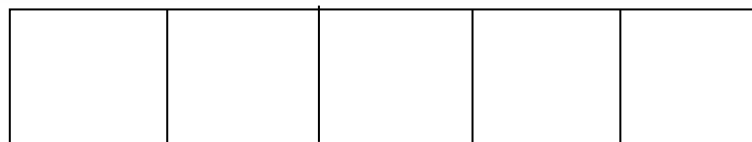
变量:

Empty=5

Full=0

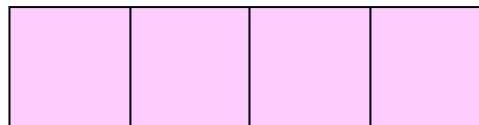
Mutex=1

缓冲区



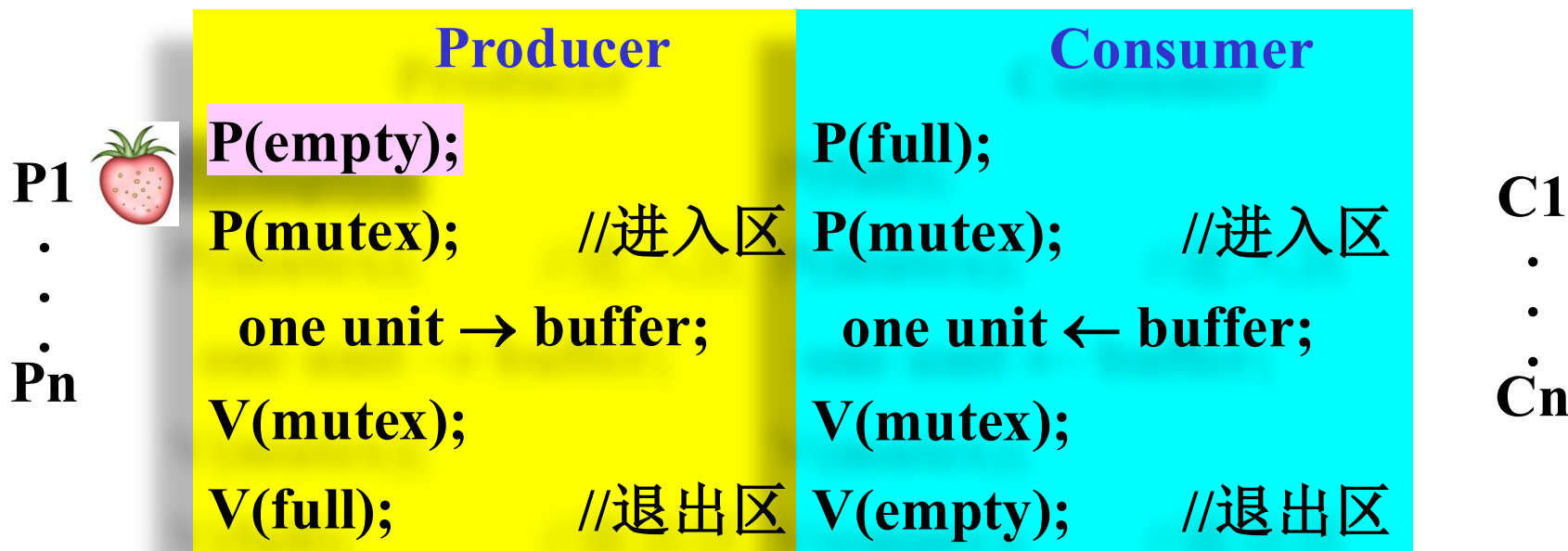
生产者进程生产一个新数据

Empty
等待队列





4.3 信号量



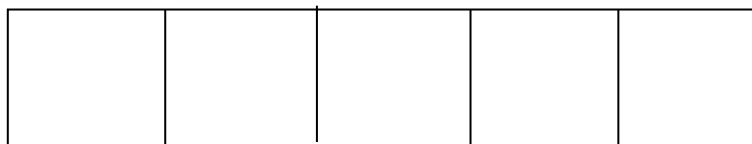
变量:

Empty=4

Full=0

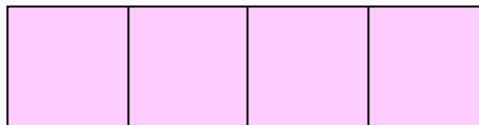
Mutex=1

缓冲区



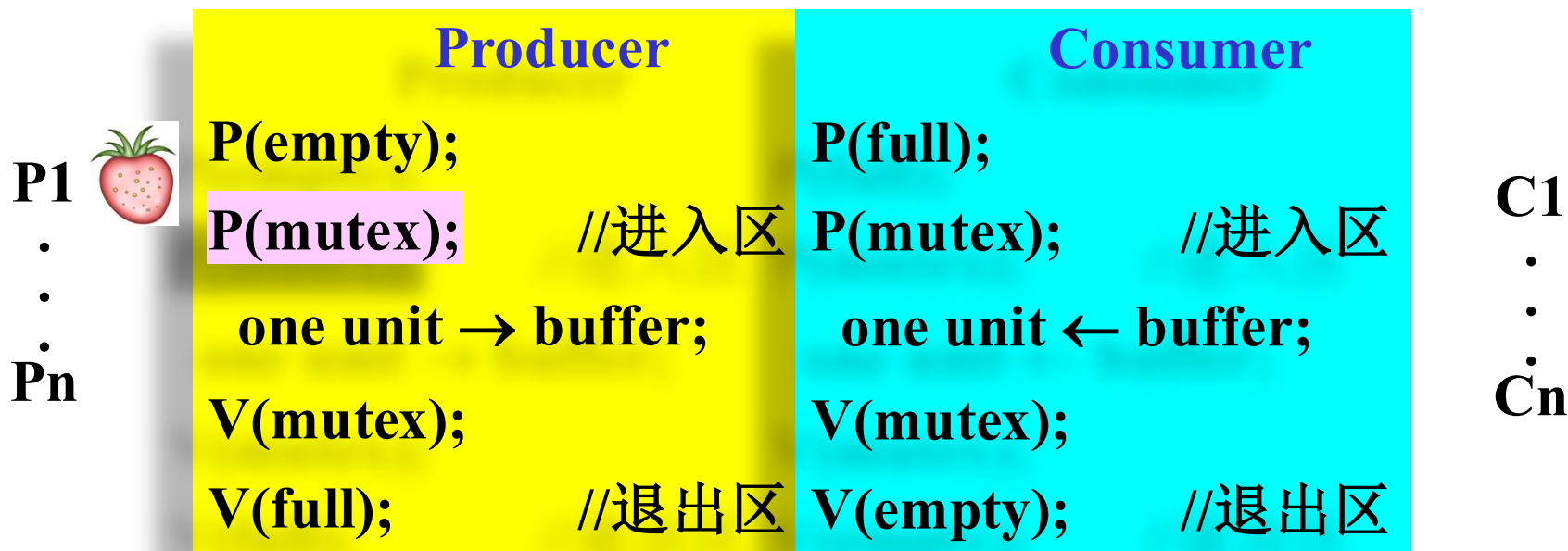
生产者进程测试一个 “empty”

Empty
等待队列





4.3 信号量



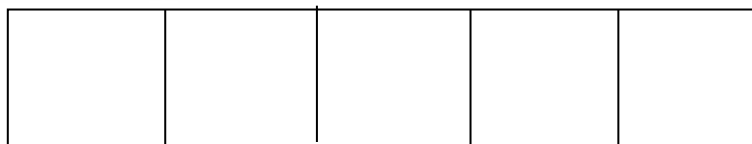
变量:

Empty=4

Full=0

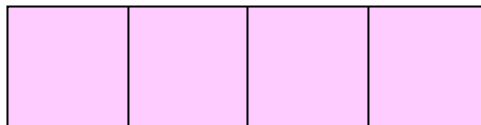
Mutex=0

缓冲区



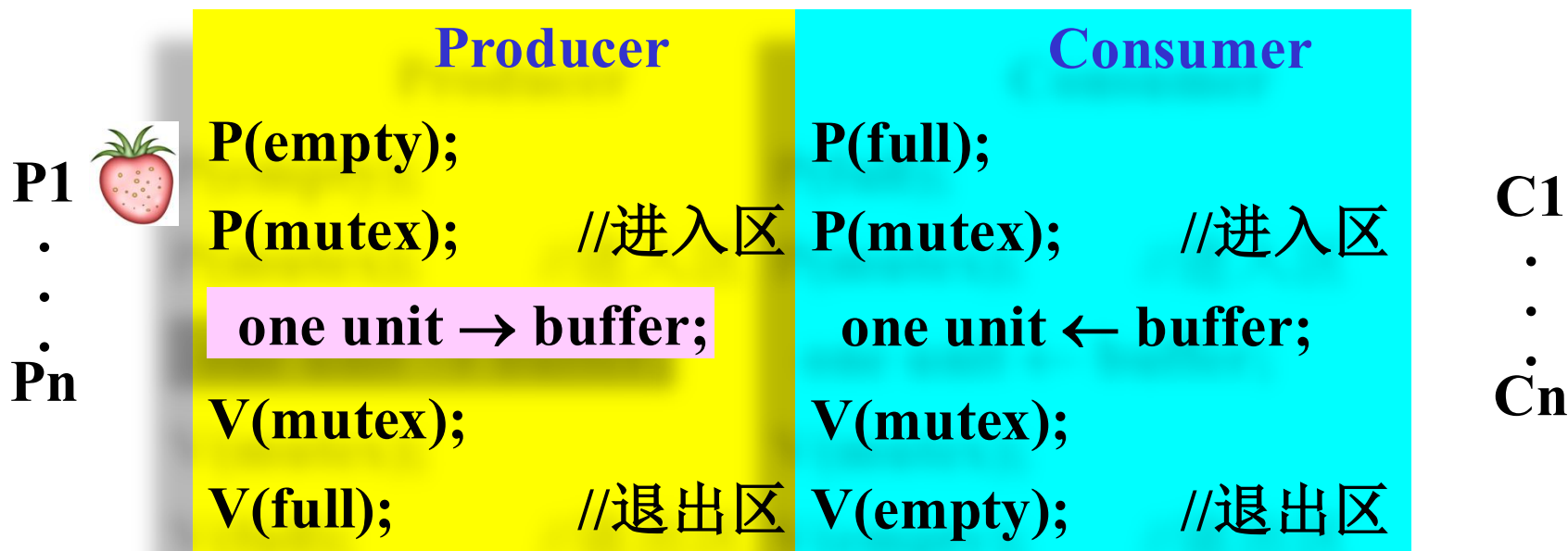
生产者进程测试临界区

Empty
等待队列





4.3 信号量



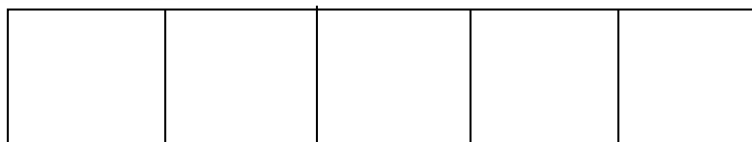
变量:

Empty=4

Full=0

Mutex=0

缓冲区



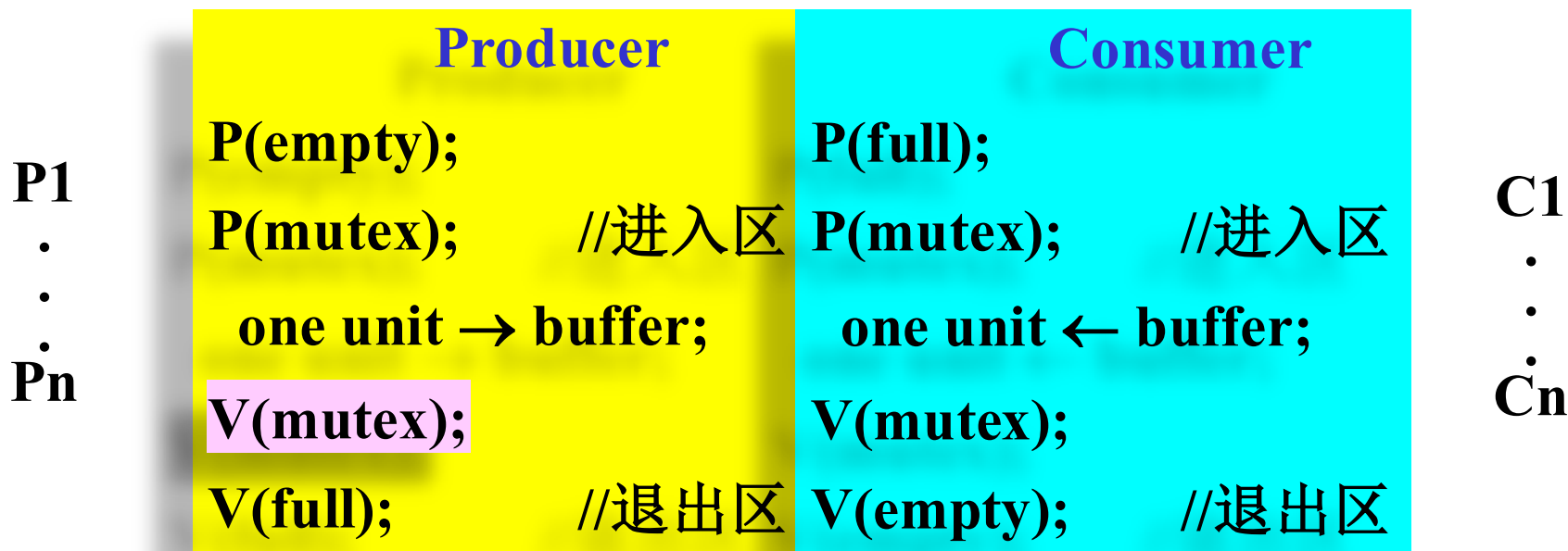
生产者进程将数据放入缓冲区

Empty
等待队列





4.3 信号量



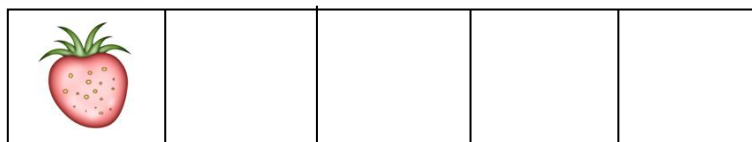
变量:

Empty=4

Full=0

Mutex=1

缓冲区



生产者进程释放临界区

Empty
等待队列





4.3 信号量

	Producer	Consumer	
P1	P(empty);	P(full);	C1
.	P(mutex); //进入区	P(mutex); //进入区	.
.	one unit → buffer;	one unit ← buffer;	.
Pn	V(mutex);	V(mutex);	Cn
	V(full); //退出区	V(empty); //退出区	

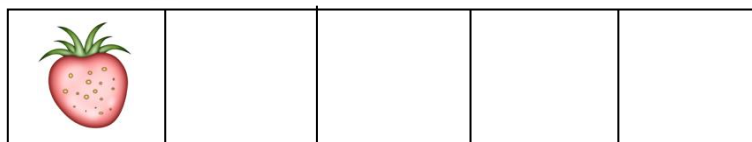
变量:

Empty=4

Full=1

Mutex=1

缓冲区



生产者进程释放一个 “full”

Empty
等待队列





4.3 信号量

	Producer	Consumer	
P1	P(empty);	P(full);	C1
.	P(mutex); //进入区	P(mutex); //进入区	.
.	one unit → buffer;	one unit ← buffer;	.
Pn	V(mutex);	V(mutex);	Cn
	V(full); //退出区	V(empty); //退出区	

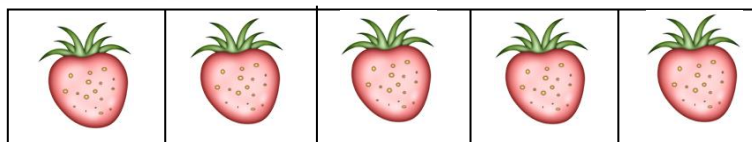
变量:

Empty=0

Full=5

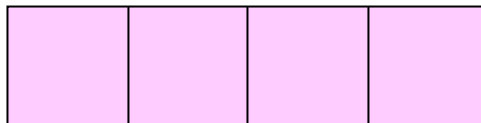
Mutex=1

缓冲区



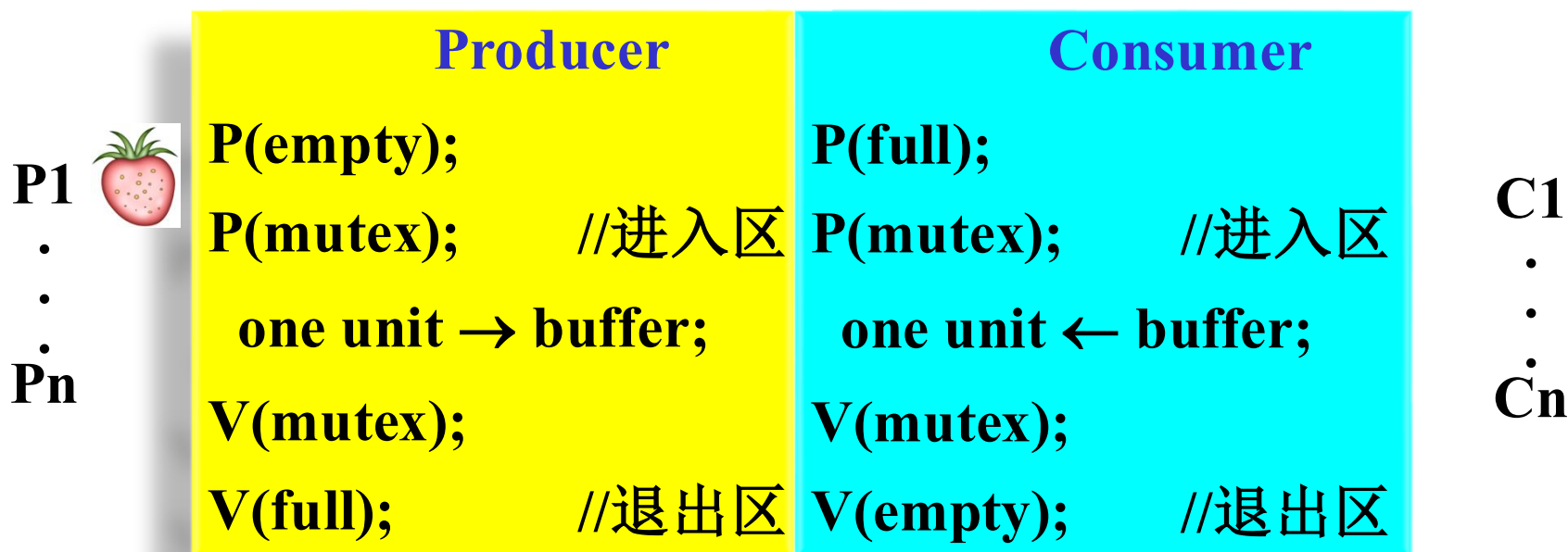
再重复4次...

Empty
等待队列





4.3 信号量



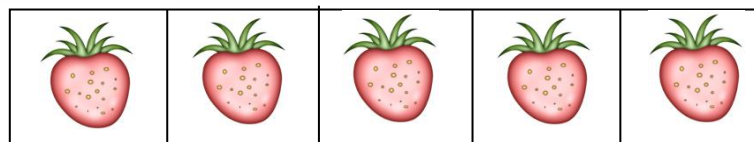
变量:

Empty=0

Full=5

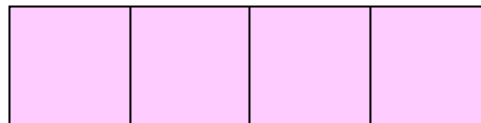
Mutex=1

缓冲区



生产者进程又生产一个新数据...

Empty
等待队列





4.3 信号量

P1 
.
.
.
Pn

Producer	Consumer
P(empty);	P(full);
P(mutex); //进入区	P(mutex); //进入区
one unit → buffer;	one unit ← buffer;
V(mutex);	V(mutex);
V(full); //退出区	V(empty); //退出区

C1
.
.
.
Cn

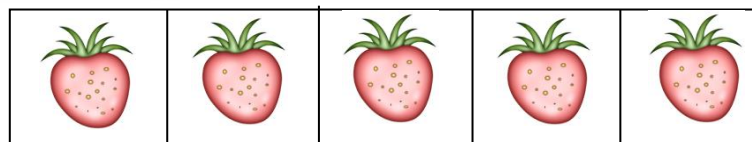
变量:

Empty=-1

Full=5

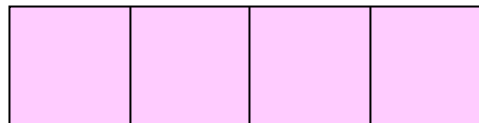
Mutex=1

缓冲区



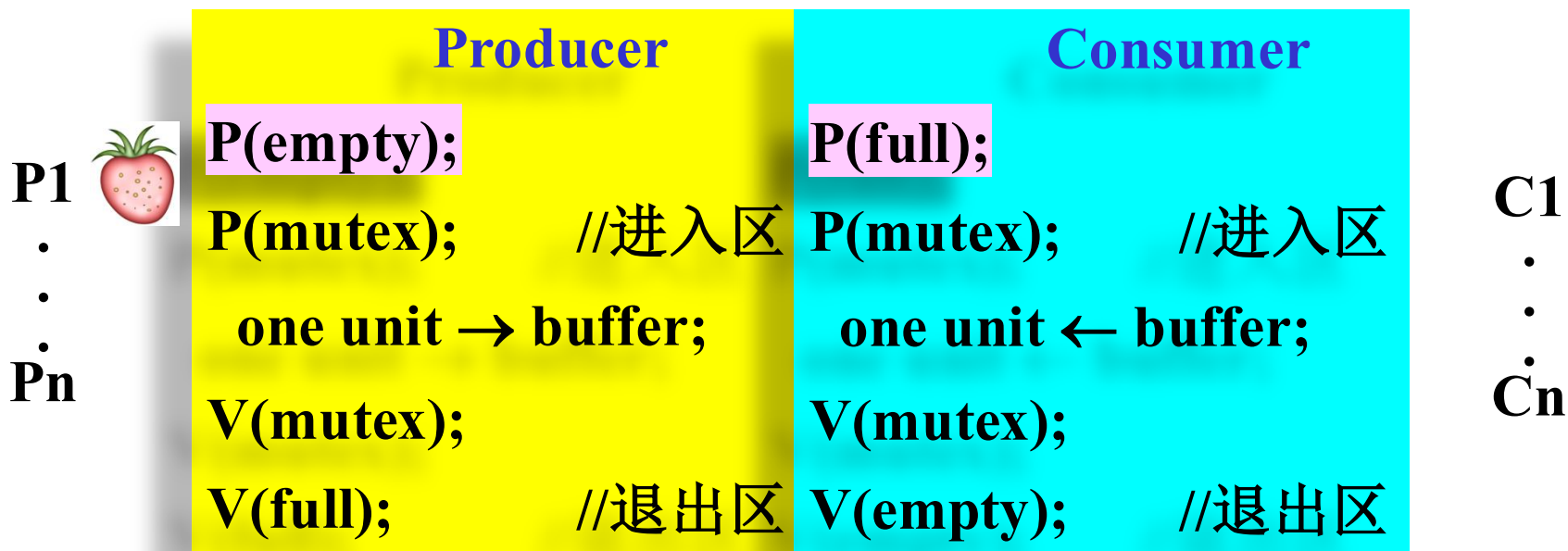
生产者进程测试一个 “empty”

Empty
等待队列





4.3 信号量



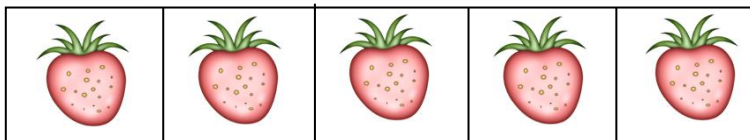
变量:

Empty=-1

Full=4

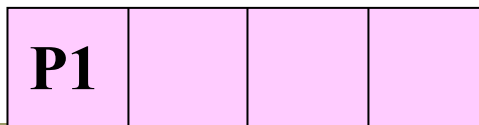
Mutex=1

缓冲区



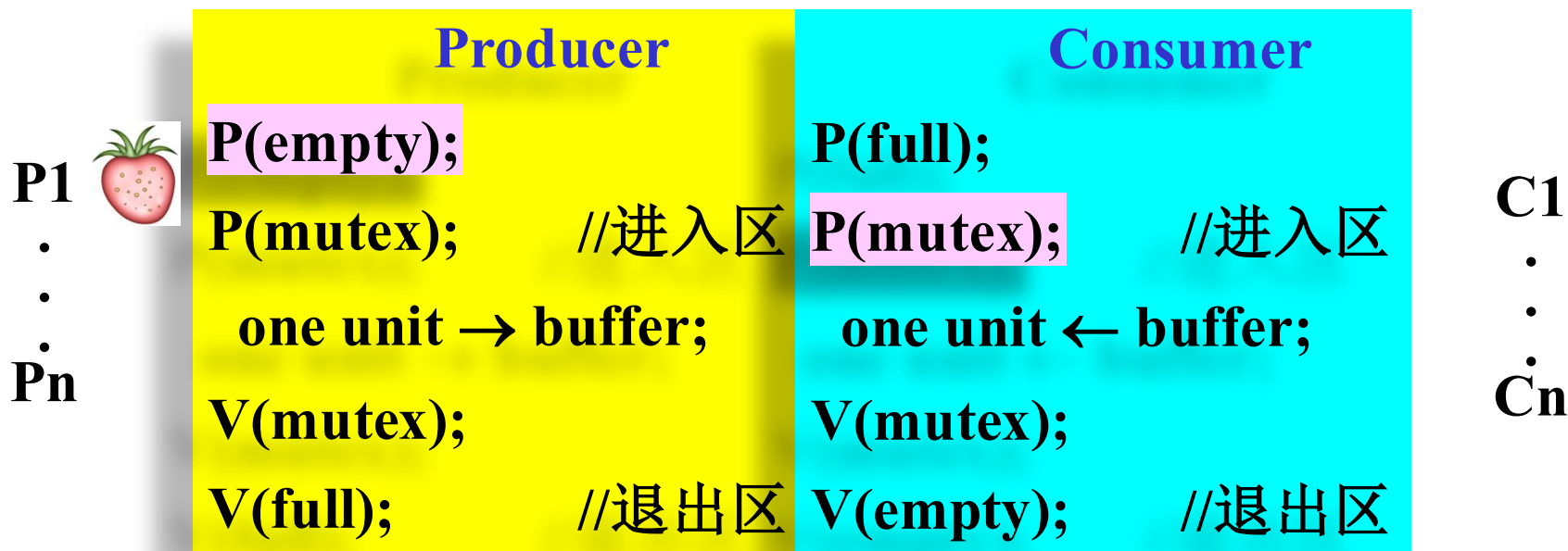
消费者进程测试一个 “full”

Empty
等待队列





4.3 信号量



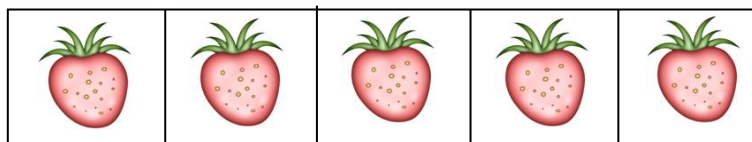
变量:

Empty=-1

Full=4

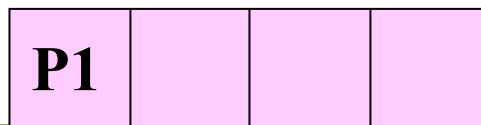
Mutex=0

缓冲区



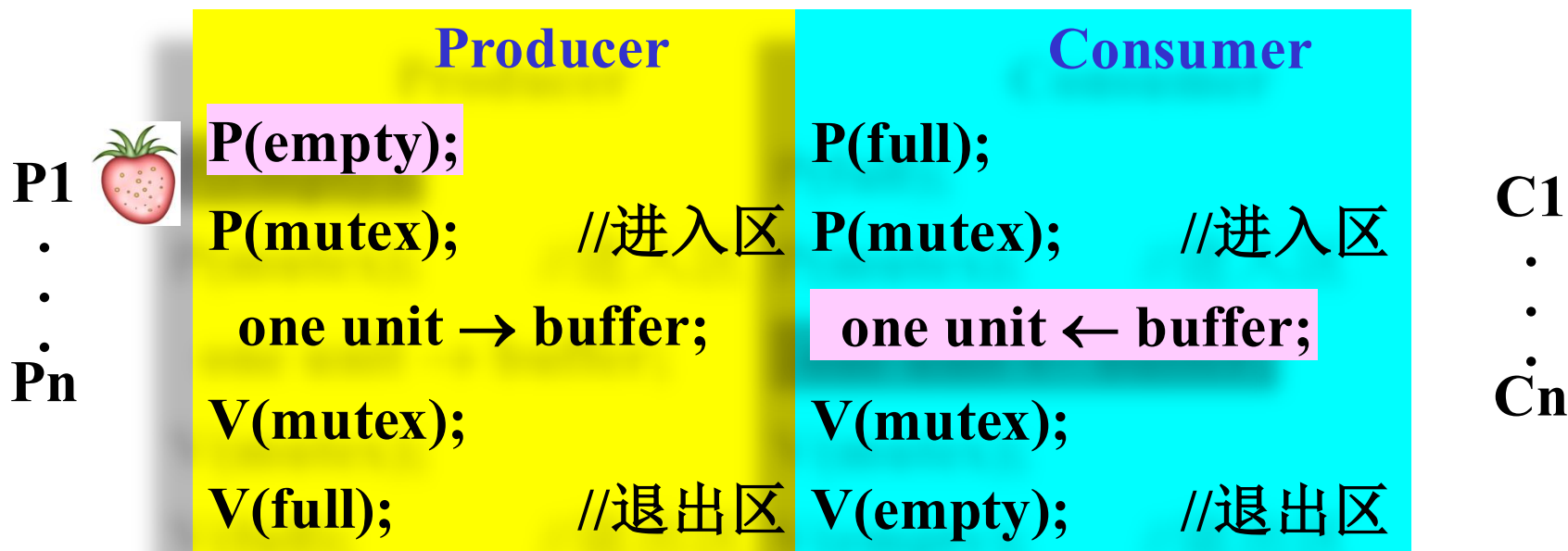
消费者进程测试临界区

Empty
等待队列





4.3 信号量



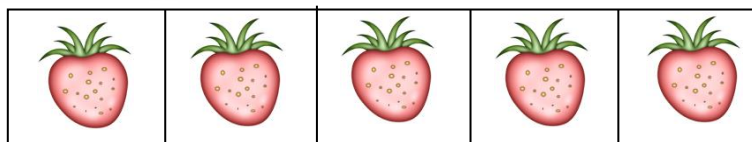
变量:

Empty=-1

Full=4

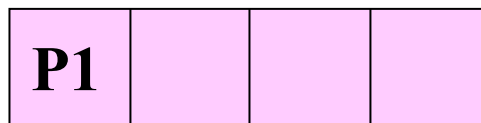
Mutex=0

缓冲区



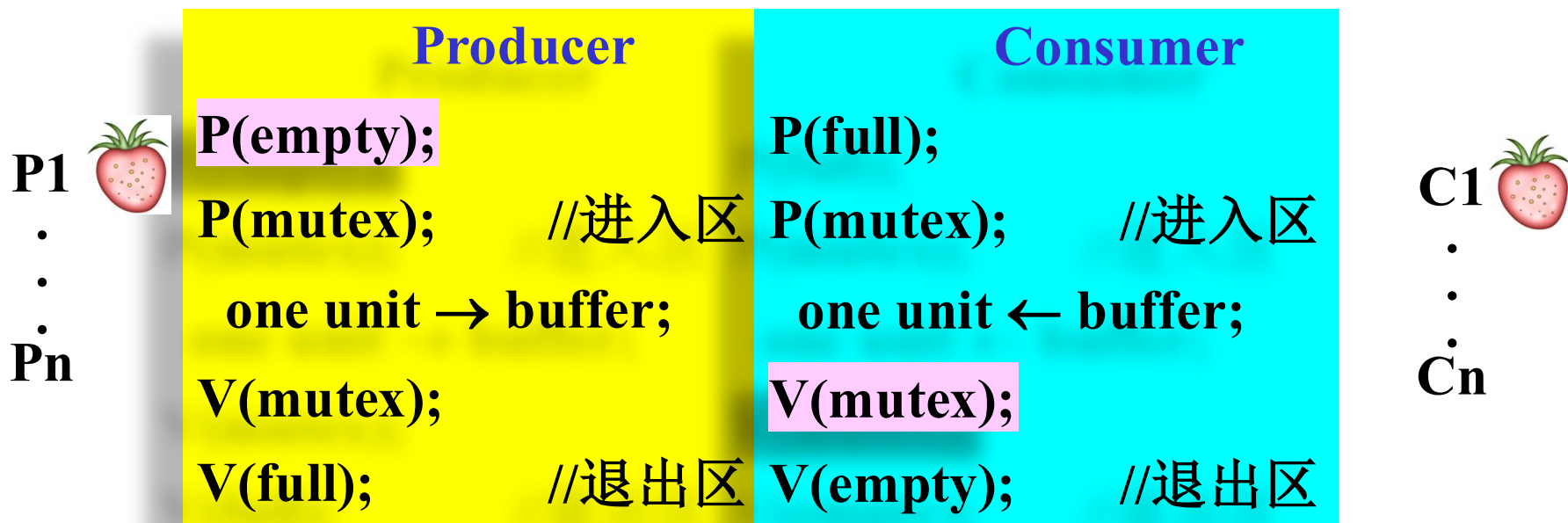
消费者进程从缓冲区获得一个数据

Empty
等待队列





4.3 信号量



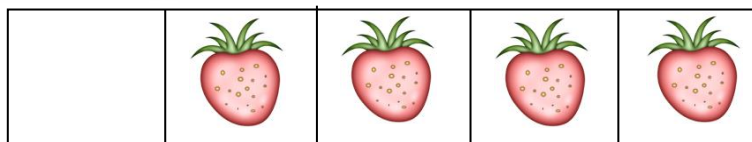
变量:

Empty=-1

Full=4

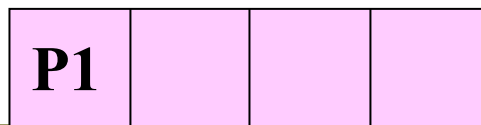
Mutex=1

缓冲区



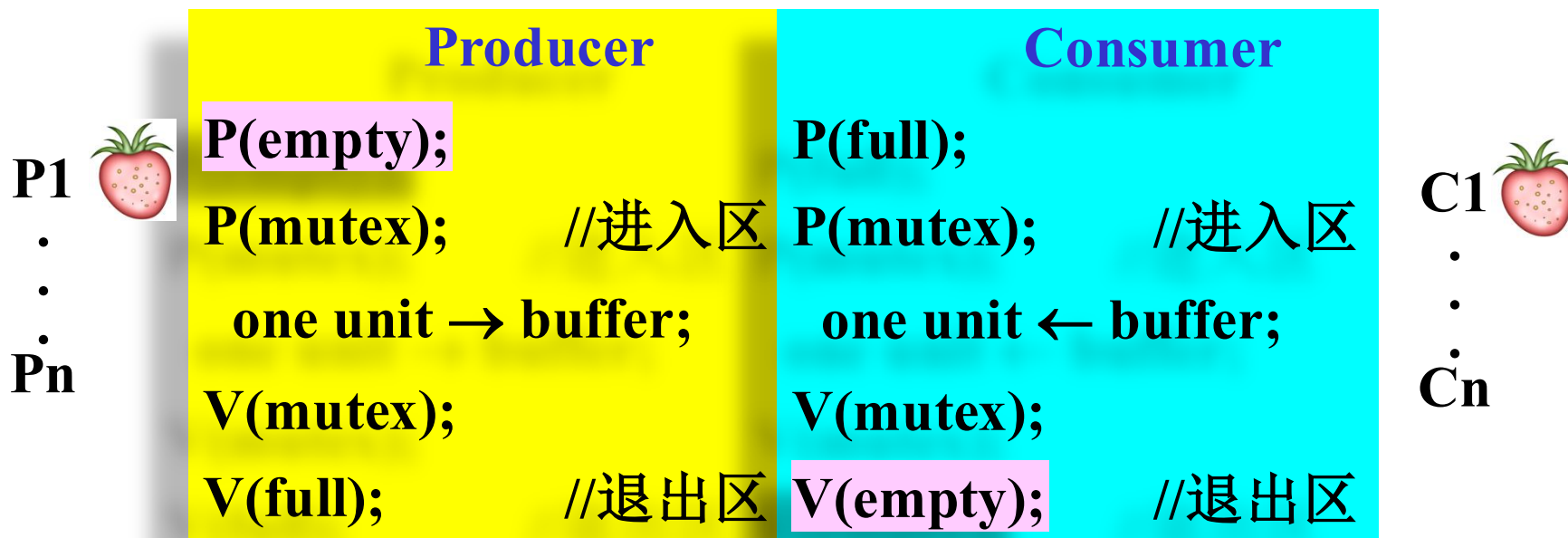
消费者进程释放临界区

Empty
等待队列





4.3 信号量



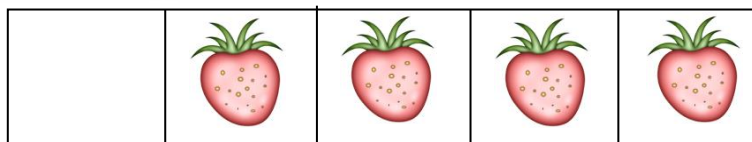
变量:

Empty=0

Full=4

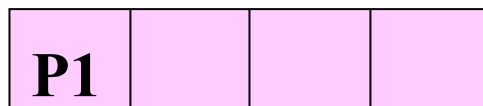
Mutex=1

缓冲区



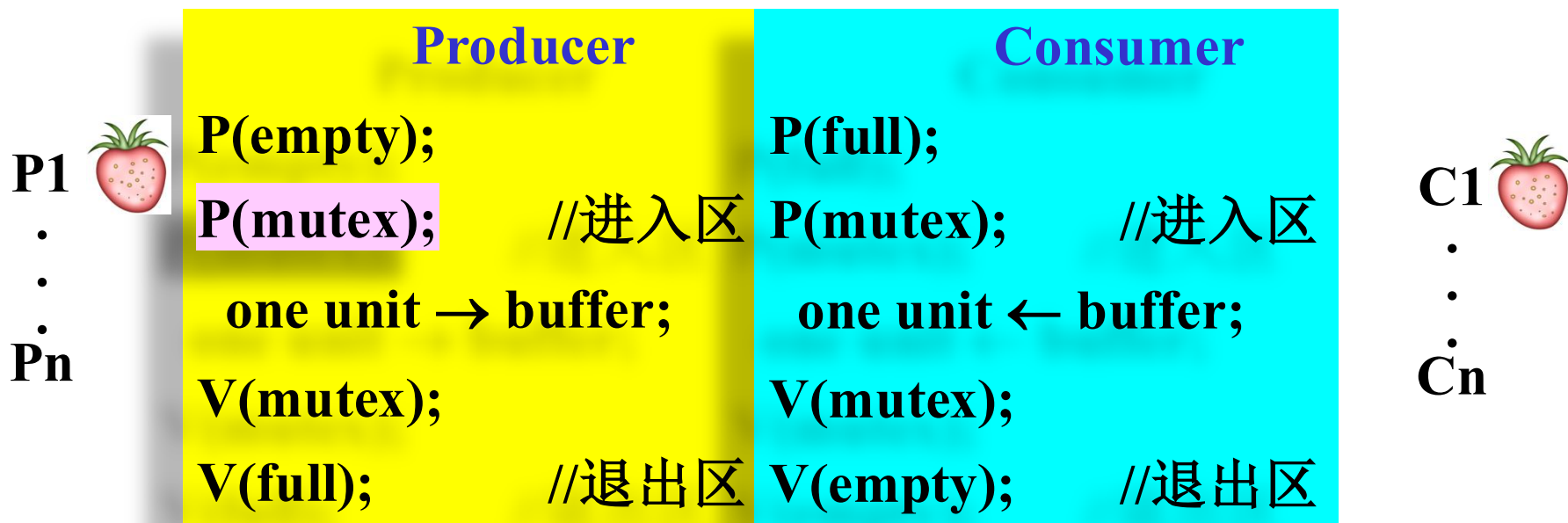
消费者进程释放一个 “empty”

Empty
等待队列





4.3 信号量



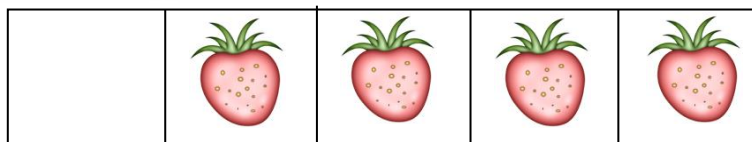
变量:

Empty=0

Full=4

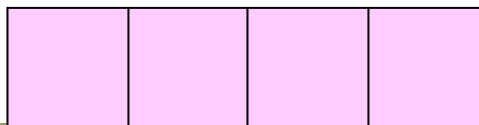
Mutex=0

缓冲区



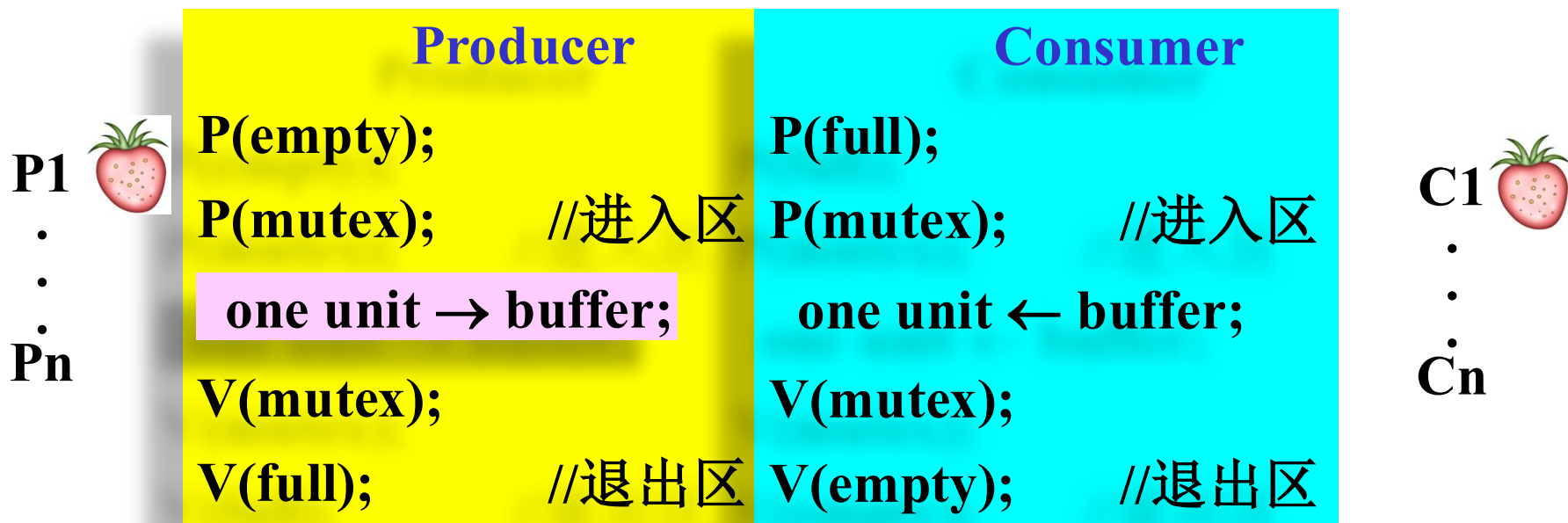
生产者进程测试临界区

Empty
等待队列





4.3 信号量



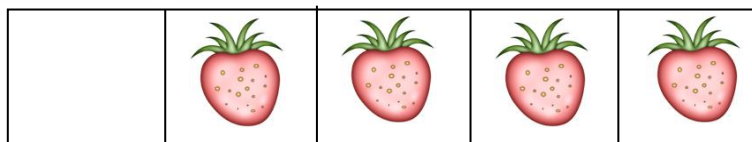
变量:

Empty=0

Full=4

Mutex=0

缓冲区



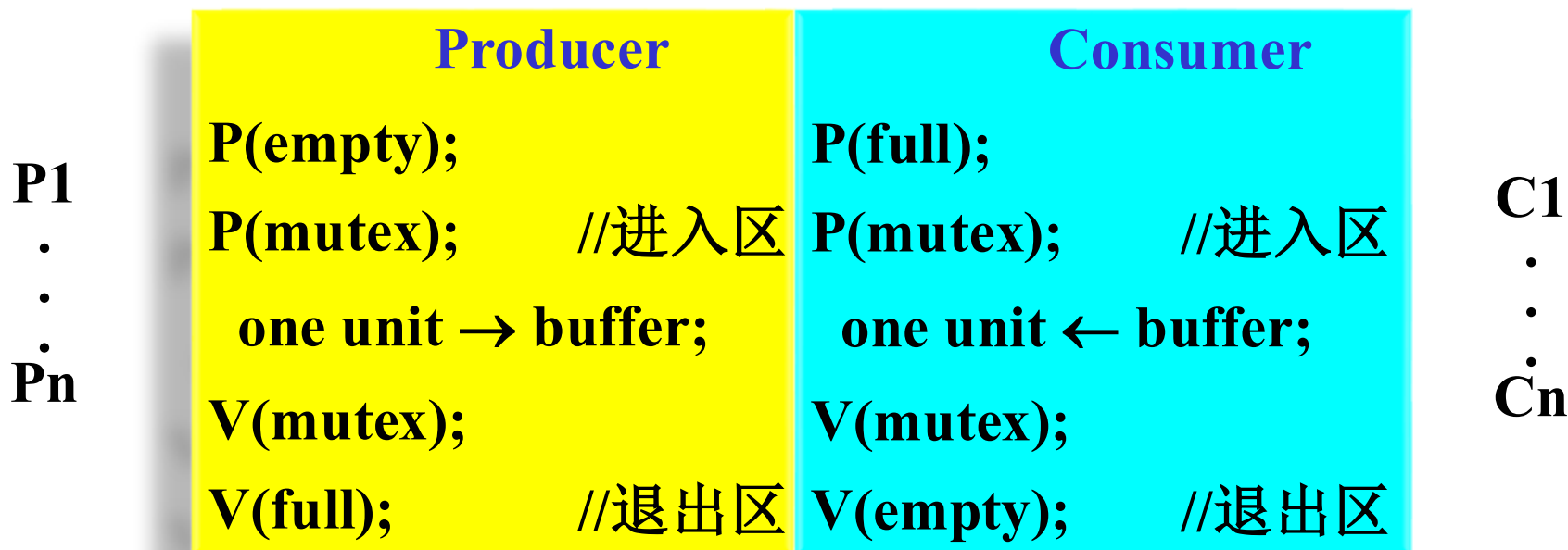
生产者进程将一个数据放入缓冲区

Empty
等待队列





4.3 信号量



变量:

Empty=0

Full=4

Mutex=0

缓冲

生产者
消费者

生产者与消费者同步

作业：以该图为基础，写（画）出缓冲区已空时，①一个消费者进程执行P(full)后；②又一个消费者进程执行P(full)后；③一个生产者进程执行V(full)后；各变量的值及等待队列的情况。



4.3 信号量

操作的顺序很重要，不当会产生死锁。
例如：假定Producer和Consumer如下：

Producer:

P(empty);

P(mutex);

one unit → buf;

V(mutex);

V(full);

Consumer:

P(mutex);

P(full); //进入区

one unit ← buf;

V(mutex);

V(empty); //退出区



讨论：这样正确吗？会发生什么？

当full=0, mutex = 1时，执行顺序：

Consumer.P(mutex) ; Consumer.P(full); // C阻塞，等待Producer发出的full信号

Producer.P(empty) ; Producer.P(mutex) ; // P阻塞，等待Cons.发出的empty信号

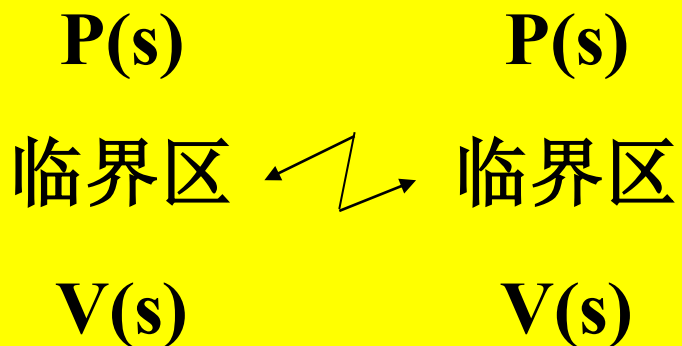
死锁！！ 还有其他的执行序列可以产生死锁吗？



4.3 信号量

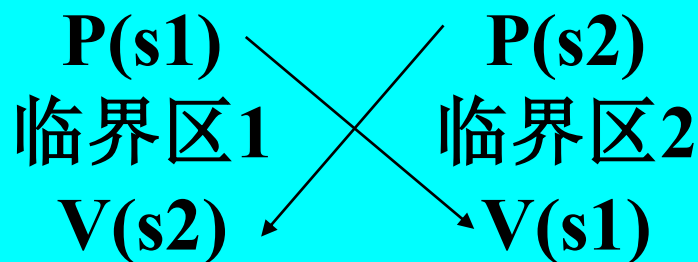
(5) 利用信号量实现进程同步和互斥

S初始值为1



互斥

S1初始值n, s2初始值0



同步

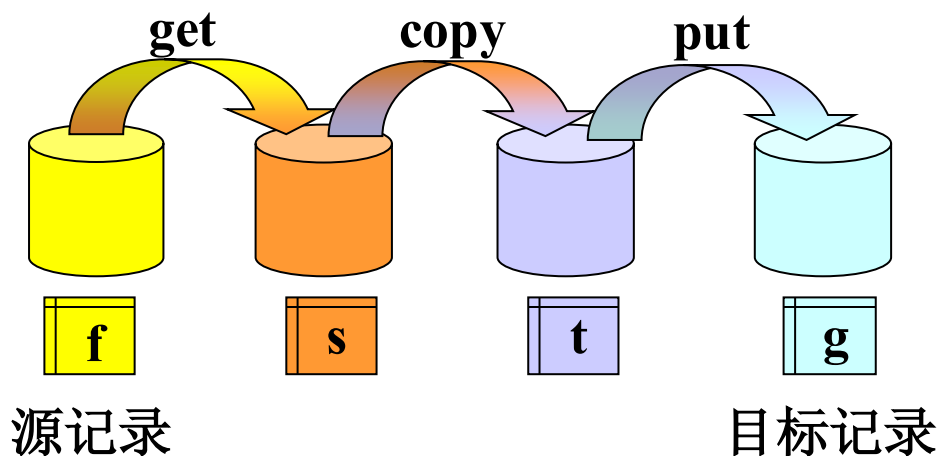
观察： 利用信号量实现互斥、同步的特点是什么？



4.3 信号量

思考与作业:

1. 上述的生产者和消费者之间是互斥的，生产者与生产者之间以及消费者与消费者之间也是互斥的，是否可以实现生产者和消费者之间的并行？如何实现？
2. 用P.V操作解决司机与售票员的问题
3. 用P.V操作解决下图之同步问题:





4.3 信号量

(6) 信号量P、V操作的实现

P(Semaphore s)

```
{  
    封锁中断;           // 对本CPU封锁中断  
    lock(lockbit); // 对临界区加锁  
    --s.count;         //表示申请一个资源;  
    if (s.count < 0) //表示没有空闲资源;  
    { 调用进程进入等待队列s.queue;  
        阻塞调用进程;  
    }  
    unlock(lockbit)  
    开放中断;  
};
```

```
typedef struct {  
    int count;  
    Pointer_PCB queue;  
} Semaphore;  
Semaphore s;
```

```
int TestAndSet(int *ptr, int new)  
{  
    int old = *ptr; // fetch old value at ptr  
    *ptr = new;     // store 'new' into ptr  
    return old;     // return the old value  
}
```

```
typedef struct __lock_t {  
    int flag;  
} lock_t;
```

```
void lock(lock_t *lock) {  
    while(TestAndSet(&lock->flag, 1) == 1);  
    // spin-wait (do nothing)  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

问题： P(s)本身是一段程序，其中实现对Semaphore结构的修改，这个修改是否需要互斥进行？如何互斥



4.3 信号量

(6) 信号量P、V操作的实现

1. **V(s)**
2. **BEGIN**
3. **封锁中断;**
4. **lock(lockbit);**
5. **++s.count;**
6. **if (s.count <= 0)**
7. **{从等待队列s.queue中取出一个进程P;**
 进程P进入就绪队列;
 }
8. **unlock(lockbit);**
9. **开放中断;**
10. **END;**



4.3 信号量



小结--take away

1. 信号量的概念：整型信号量、记录型信号量
2. P、V操作的操作逻辑、实现
3. 利用信号量实现互斥、同步的模式
4. 生产者消费者问题**



4.4 经典进程同步问题

(1) 读者—写者问题 (the readers-writers problem)

● **问题描述：**对共享资源的读写操作，任一时刻“写者”最多只允许一个，而“读者”则允许多个

- “读—写” 互斥
- “写—写” 互斥
- “读—读” 允许



4.4 经典进程同步问题

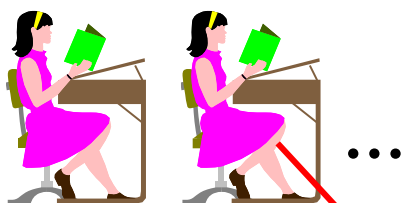


思路：进入临界区

情景1：若干读者

情景2：一个写者

情景3：无读者、写者



阻塞写者

阻塞读者、写者

可以读

阻塞

阻塞

进入

可以读，阻止写

判断读者数量： >0 读者加1； $=0$ 判断可读否，可则读者加1

阻塞



判断可写否，可则写

可以写，阻止读、写

● 第一个读者判断是否有写者



4.4 经典进程同步问题



思路：退出临界区

情景1：若干读者



读者数量减1

情景2：只剩下一个读者



读者数量减1，唤醒写者

情景3：一个写者



释放读者、写者

● 最后一个读者释放写者



4.4 经典进程同步问题

4.4.1 读者—写者问题 (the readers-writers problem)

● 分析

采用信号量机制:

- **Wmutex**表示“允许写进入”，初值是1。
- 公共变量**Rcount**表示“正在读”的进程数，初值是0；
- **Rmutex**表示对**Rcount**的互斥操作，初值是1。

读者进入时: 只要一个reader进程在读，便不允许writer进程去写。
仅当Rcount=0, reader进程才需要执行P(Wmutex)操作,

reader离开时: 仅当reader进程执行了Rcount减1操作后,
其值为0时, 才须执行V(Wmutex)操作



4.4 经典进程同步问题

4.4.1 读者—写者问题the readers-writers problem

Writer

```
P(Wmutex);  
write;  
V(Wmutex);
```

Reader

```
P(Wmutex);  
  
read;  
  
V(Wmutex);
```



4.4 经典进程同步问题

经典进程同步问题——思考

- 分析该读者/写者问题，对读者和写者是否公平？

```
P(Wmutex);  
write;  
V(Wmutex);
```



```
P(Rmutex)  
If (Rcount==0)  
  P(Wmutex);  
  ++Rcount;  
V(Rmutex);  
read;  
P(Rmutex);  
--Rcount;  
If (Rcount==0)  
  V(Wmutex);  
V(Rmutex);
```

● 分析：

- 这样实现是否合适？**存在什么问题不？**
- 假设读者源源不断地到达，会是什么情况？
- 只要临界区中有读者，其他读者便可读，可能造成**写者饥饿**，**对读者不公平**



4.4 经典进程同步问题

4.4.2 第二类读者-写者问题【自学】

- 第二类读者写者问题(并分析对读者和写者是否公平), 条件:
 - 多个读者可以**同时进行读**;
 - **写者必须互斥** (只允许一个写者写, 也不能读者、写者同时进行);
 - **写者优先于读者** (一旦有写者, 则后续读者必须等待, **唤醒时优先考虑写者**)。
- 给出对读者/写者都公平的关于读者写者问题的描述



4.4 经典进程同步问题

4.4.2 第二类读者-写者问题

writer:

```
P(y);  
If Wcount==0  
    P(Rmutex);  
    ++Wcount;  
V(y);  
P(Wmutex);  
write;  
V(Wmutex);  
P(y);  
--Wcount;  
If (Wcount==0)  
    V(Rmutex)  
V(y);
```

- Wcount为正
在写和提出写
要求的进程数，
初值为0

- Rcount为正在
读和提出读的
进程数，初值
为0。

- Rmutex为读、
写互斥信号量，
初值为1。

- Wmutex为写
互斥信号量，
初值为1。



Reader:

```
P(z)  
P(Rmutex);  
P(x)  
If (Rcount==0)  
    P(Wmutex);  
    ++Rcount;  
V(x);  
V(Rmutex);  
V(z)  
read;  
P(x);  
--Rcount;  
If (Rcount==0)  
    V(Wmutex);  
V(x);
```

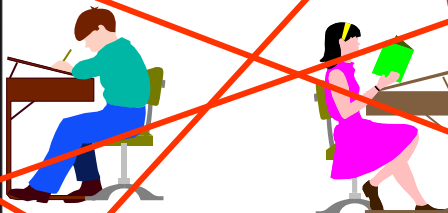


4.4 经典进程同步问题

4.4.2 第二类读者-写者问题

writer:

```
P(y);  
If Wcount==0  
    P(Rmutex);  
    ++Wcount;  
V(y);  
P(Wmutex);  
write;  
V(Wmutex);  
P(y);  
--Wcount;  
If (Wcount==0)  
    V(Rmutex)  
V(y);
```



Reader:

```
P(z);  
P(Rmutex);  
P(x)  
If (Rcount==0)  
    P(Wmutex);  
    ++Rcount;  
V(x);  
V(Rmutex);  
V(z)  
read;  
P(x);  
--Rcount;  
If (Rcount==0)  
    V(Wmutex);  
V(x);
```




4.4 经典进程同步问题

4.4.2 第二类读者-写者问题

writer:

```
P(y);  
If Wcount==0  
    P(Rmutex);  
    ++Wcount;  
V(y);  
P(Wmutex);  
write;  
V(Wmutex);  
P(y);  
--Wcount;  
If (Wcount==0)  
    V(Rmutex)  
V(y);
```

Reader:

```
P(z);  
P(Rmutex);  
P(x)  
If (Rcount==0)  
    P(Wmutex);  
    ++Rcount;  
V(x);  
V(Rmutex);  
V(z)  
read;  
P(x);  
--Rcount;  
If (Rcount==0)  
    V(Wmutex);  
V(x);
```

1. 系统中只有读者进程

- 第一个读者进程
 - ✓ 执行P(Wmutex)
 - ✓ 没有队列

2. 系统中只有写者进程

- 第一个写者进程执行P(Rmutex)
- 所有后续写者:
 - ✓ 均执行P(Wmutex)
 - ✓ 写者进程在Wmutex上排队



4.4 经典进程同步问题

4.4.2 第二类读者-写者问题

writer:

```
P(y);  
If Wcount==0  
    P(Rmutex);  
    ++Wcount;  
V(y);  
P(Wmutex);  
write;  
V(Wmutex);  
P(y);  
--Wcount;  
If (Wcount==0)  
    V(Rmutex)  
V(y);
```

Reader:

```
P(z);  
P(Rmutex);  
P(x)  
If (Rcount==0)  
    P(Wmutex);  
    ++Rcount;  
V(x);  
V(Rmutex);  
V(z)  
read;  
P(x);  
--Rcount;  
If (Rcount==0)  
    V(Wmutex);  
V(x);
```

3. 系统中读者、写者进程均有，读者进程在临界区

- 第一个读者进程：
 - ✓ 执行P(Wmutex)
- 第一个写者进程：
 - ✓ 执行P(Rmutex)
- 所有写者进程：
 - ✓ 在Wmutex上排队
- 第一个未进入临界区的读者：
 - ✓ 在Rmutex上排队
- 其它未进入临界区的读者：
 - ✓ 在z上排队



4.4 经典进程同步问题

4.4.2 第二类读者-写者问题

writer:

```
P(y);  
If Wcount==0  
  P(Rmutex);  
  ++Wcount;  
V(y);  
P(Wmutex);  
write;  
V(Wmutex);  
P(y);  
--Wcount;  
If (Wcount==0)  
  V(Rmutex)  
V(y);
```

Reader:

```
P(z);  
P(Rmutex);  
P(x)  
If (Rcount==0)  
  P(Wmutex);  
  ++Rcount;  
V(x);  
V(Rmutex);  
V(z)  
read;  
P(x);  
--Rcount;  
If (Rcount==0)  
  V(Wmutex);  
V(x);
```

4:系统中读者、写者进程均有, 写者进程在临界区

- 第一个写者进程执行 P(Rmutex)
- 每个欲进入临界区的写者均执行 P(Wmutex)
- 未进入临界区的写者进程在 Wmutex 上排队
- 第一个读者进程在 Rmutex 上排队
- 其它读者进程在 z 上排队



Review



- 生产者消费者问题
 - 同步、互斥问题如何分析
 - 初值如何设置
 - P操作的顺序很重要，V操作顺序可换位置
 - 互斥的粒度：粒度粗，并发性差；粒度细并发度高
 - 非常典型，希望理解下的背记
- 读者-写者问题
 - 分析过程：各种情景怎么做，“第一人”的责任
 - 公平性分析：第二类读者写者问题
 - 非常典型，希望理解下的背记
- 使用信号量解决同步问题的一般方法是什么？
 - 先分析好需求，并用伪代码描述功能性流程，暂不考虑同步，然后再根据同步和互斥，然后设立信号量，添加P、V

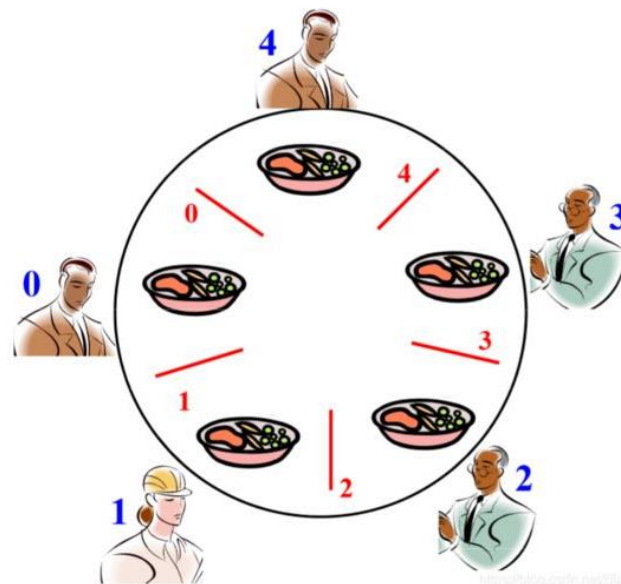


4.4 经典进程同步问题

4.4.3 哲学家进餐问题the dining philosophers problem

🌐 问题描述：（由Dijkstra首先提出并解决）

- 5个哲学家围绕一张圆桌而坐；
- 桌子上放着5支筷子，每两个哲学家之间放一支；
- 哲学家的动作包括思考和进餐，
- 进餐时需要同时拿起他左边和右边的两支筷子；
- 思考时则同时将两支筷子放回原处。





4.4 经典进程同步问题

4.4.3 哲学家进餐问题the dining philosophers problem

- 哲学家的生活规律

Repeat

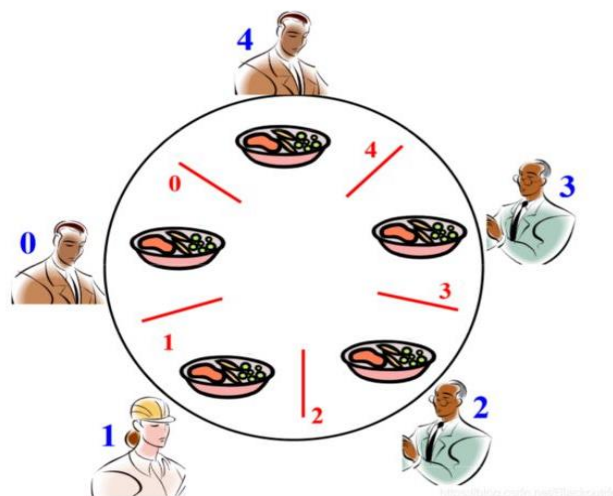
思考;

取chop[i]; 取chop[(i+1) mod 5];

进食;

放chop[i]; 放chop[(i+1) mod 5];

Until false;



- 思考：这里什么是临界资源？

- 问题：如何保证哲学家们的动作有序进行？如：

- ① 不出现相邻者同时要求进餐；

- ② 不出现有人永远拿不到筷子；



4.4 经典进程同步问题

4.4.3 哲学家进餐问题the dining philosophers problem

- 一个信号量表示一根筷子，五个信号量构成信号量组 chop[5]，所有信号量初始值为1。
- Phi: 第i个哲学家的生活描述为：

```
Repeat
    思考问题
    P(chop[i]);
    P(chop[(i+1) mod 5]);
    进餐
    V(chop[i]);
    V(chop[(i+1) mod 5]);
Until False
```

- ✓ **思考：**该实现怎么样？存在什么问题？解决了上面的两个问题了吗？
- ✓ **分析：**该算法可保证两个相邻的哲学家不能同时进餐，但不能防止五位哲学家同时拿起各自左边的筷子、又试图拿起右边的筷子，这会引起**死锁**



4.4 经典进程同步问题

4.4.3 哲学家进餐问题the dining philosophers problem

- 为防止死锁发生可采取的措施

- ① 最多允许4个哲学家同时坐在桌子周围 (√) ;
 - ✓ 产生了什么变化?
 - ✓ 同时吃饭的人数, 就餐的座位数成了紧俏资源
- ② 仅当一个哲学家左右两边的筷子都可用时, 才允许他拿筷子 (√) ;
 - ✓ 怎么办? 测试test
- ③ 给所有哲学家编号, 奇数号的哲学家必须首先拿左边的筷子, 偶数号的哲学家则反之;
- ④ 为了避免死锁, 把哲学家分为三种状态, 思考, 饥饿, 进食, 并且一次拿到两只筷子, 否则不拿。



4.4 经典进程同步问题

Program diningphilosophers

Semaphore chop[5] = {1};

Semaphore room = 4;

```
Void philosopher (int i) {  
    while (true) {  
        thinking();  
        P(room);  
        P(chop[i]);  
        P(chop[(i+1) mod 5])  
        eating();  
        V(chop[i]);  
        V(chop[(i+1) mod 5])  
        V(room); }  
}
```





4.4 经典进程同步问题

4.4.3 哲学家就餐问题

- 哲学家就餐问题的另一种解法，请分析其实现互斥的过程：
- 变量设置：
state: array[0..4] of (思考, 饥饿, 进食) ;
ph: array[0..4] of semaphore;
mutex: semaphore;
i: 0..4;

后跳

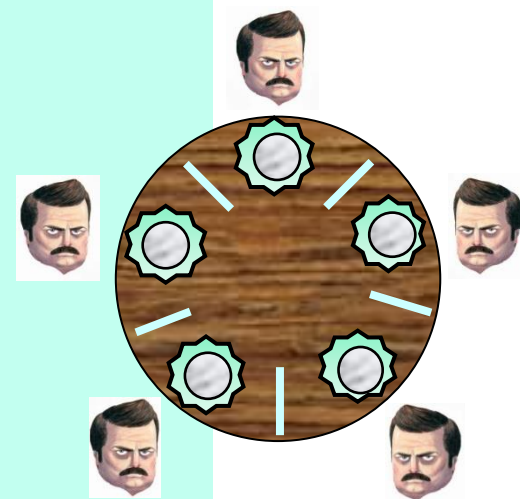


4.4 经典进程同步问题

4.4.3 哲学家就餐问题

- 哲学家就餐问题的另一种解法，请分析其实现互斥的过程：

```
Procedure test(i:=0...4);  
begin  
  if (state[i] = 饥饿) And  
    (state[(i-1)mod5] != 进食) And  
    (state[(i+1)mod5] != 进食)  
  then begin  
    state[i] = 进食  
    V(ph[i])  
  end  
end
```





4.4 经典进程同步问题

Procedure philosopher(i: 0~4)

Begin

Repeat

思考;

state[i]:=饥饿;

P(mutex);

test(i);

V(mutex);

P(ph[i]);

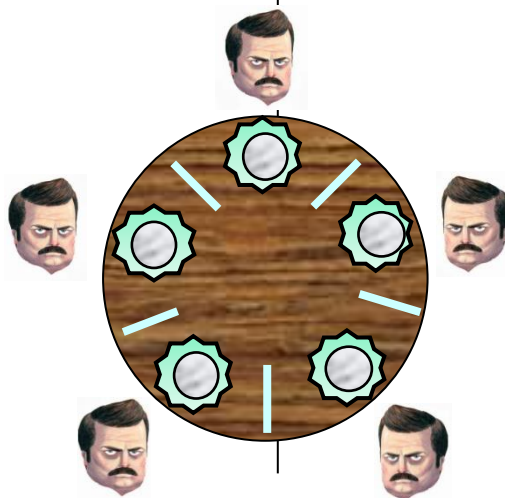
拿左筷子;

拿右筷子;

进食;

放左筷子;

放右筷子;



Procedure test(i:=0...4);

begin

if (state[i] = 饥饿) And

(state[(i-1)mod5] <> 进食) And

(state[(i+1)mod5] <> 进食)

then begin

state[i]=进食

V(ph[i])

end

end

P(mutex)

state[i]:=思考;

test([(i-1)mod5]);

tset([(i+1)mod5]);

V(mutex);

until fails

End

初始值:

state[i]=思考

ph[i]=0

Mutex=1



4.5 信号量集

- 用于同时需要多个资源时的信号量操作
 - AND型信号量集
 - 一般信号量集



4.5 信号量集

4.5.1 AND型信号量集

- **需求：**用于**同时需要多种资源**且每种占用一个时的信号量操作
- **解决的问题：**一段处理代码需要同时获取两个或多个临界资源——**可能死锁**：例如，各进程分别获得部分临界资源，然后等待其余的临界资源，“各不相让”

Process A:
P(Dmutex);
P(Emutex);

Process B:
P(Emutex);
P(Dmutex);

问题：为什么可能会死锁？



4.5 信号量集



4.5.1 AND型信号量集

- **基本思想：** 将一段代码同时需要的多个临界资源，要么全部分配给它，要么一个都不分配
- **Swait如下：**

SP

```
Swait (S1,S2,...,Sn){  
    while (true){  
        if (S1>=1 && S2>=1 && ... && Sn>=1){  
            for (i=1; i<=n; ++i) --Si;  
            break;  
        }  
        else{  
            调用进程进入第一个小于1信号量Sj的等待队列Sj.queue;  
            阻塞调用进程;  
        }  
    }  
}
```



4.5 信号量集

4.5.1 AND型信号量集

- **基本思想：** 将一段代码同时需要的多个临界资源，要么全部分配给它，要么一个都不分配
- **Ssignal**描述如下：

```
Ssignal (S1,S2,...,Sn){  
    for (i=1; i<=n; ++i){  
        ++Si;  
        for (each process P waiting in Si.queue){  
            从等待队列Si.queue中取出进程P;  
            if (进程P通过Swait的测试)  
                进程P进入就绪队列  
            else  
                进程P进入其等待队列;  
        }  
    }  
}
```

SV



4.5 信号量集

4.5.2 一般信号量集

- **需求：**用于同时需要多种资源、每种占用的数目不同、且可分配的资源还存在一个临界值时的处理；
- **基本思想：**在AND型信号量集的基础上进行扩充：
进程对信号量 $semi$ 的测试值为 ti ，占用值为 di
- $Swait(sem1, t1, d1; \dots; semn, tn, dn)$ (或SP)
- $Ssignal(sem1, d1; \dots; semn, dn)$ (或SV)



4.5 信号量集

4.5.2 一般信号量集

● **Swait**(sem₁, t₁, d₁; ...; sem_n, t_n, d_n) (或SP)

If (sem₁ ≥ t₁ and sem₂ ≥ t₂ and ... and sem_n ≥ t_n) then

for i=1 to n do s_i := s_i - d_i

Else 阻塞进程;

● **Ssignal**(s₁, d₁; ...; s_n, d_n) (或SV)

for i=1 to n do

Begin

s_i := s_i + d_i;

for all 被阻塞在s_i上的P_i do 唤醒进程P_i;

End;

思考：这个表达式(sem₁ ≥ t₁ and sem₂ ≥ t₂ and ... and sem_n ≥ t_n), 还可以怎么表示?



4.5 信号量集

4.5.2 一般信号量集

● 一般信号量集的几种特定情况

- $Swait(S, d, d)$ ($SP(S, d, d)$) : 表示每次申请 d 个资源，当少于 d 个时，便不分配
- $Swait(S, 1, 1)$ ($SP(S, 1, 1)$) : 表示一般的记录型信号量或互斥信号量
- $Swait(S, 1, 0)$ ($SP(S, 1, 0)$) : 作为一个可控开关
 - 当 $S \geq 1$ 时，允许多个进程进入临界区
 - 当 $S = 0$ 时，禁止任何进程进入临界区



4.5 信号量集

4.5.3 信号量集应用

● 利用AND信号量解决生产者—消费者问题

- 用SP(empty, mutex)代替P(empty)和P(mutex), 用SV(mutex, full)代替V(mutex)和V(full)

生产者:



SP(empty, mutex)

One unit → buffer

SV(mutex, full)

消费者:



SP(full, mutex)

One unit ← buffer

SV(mutex, empty)



4.5 信号量集



4.5.3 信号量集应用

● 利用一般信号量集解决读者—写者问题

- ✓ 增加一个限制，即最多允许RN个读者同时读。为此引入一信号量L，其初值为RN。mx初值为1

读者：

$SP(L, 1, 1; mx, 1, 0)$

read

$SV(L, 1)$



写者：

$SP(mx, 1, 1; L, RN, 0)$

write

$SV(mx, 1)$



- $Swait(mx, 1, 0)$ 语句起着"开关"的作用。只要无writer进程进行写操作，mx就为1，reader进程就可以读。但只要有writer进程进行写操作， $mx=0$ ，任何reader进程就都无法进行读操作。
- $Swait(mx, 1, 1; L, RN, 0)$ 语句表示，仅当既无writer进程在写 ($mx=1$)，又无reader进程在读 ($L=RN$) 时，writer进程才能进入临界区写。



4.5 信号量集

4.5.3 信号量集应用-自行车装配问题-1

问题描述： 设自行车生产线上有一只箱子，其中有 N 个位置($N \geq 3$)，每个位置可存放一个车架或一个车轮；又设有三个工人，其活动分别为：

工人1：

do {

加工一个车架；

车架放入箱中；

}while(1)

工人2：

do {

加工一个车轮；

车轮放入箱中；

}while(1)

工人3：

do {

取箱中一车架；

取箱中二车轮；

组装为一台车；

}while(1)

需求： 试用信号量与wait、signal（或P、V）操作实现三个工人的合作。



4.5 信号量集

4.5.3 信号量集应用

● 利用一般信号量集解决三个工人加工自行车问题

- ✓ $\text{empty} = N$, 空位置数目
- ✓ $\text{fm} = N - 2$, 车架的最大数量
- ✓ $\text{wm} = N - 1$, 车轮的最大数量
- ✓ $f = 0$, 车架数
- ✓ $w = 0$, 车轮数

反思:

- fm 、 wm 限制各自进程的并发数
- 一个信号量只管一件事

车架工人1:



```
Do {  
    加工一个车架;  
    SP(fm,1,1; empty,1,1);  
    车架放入箱中;  
    SV(f,1);  
} while (True)
```

车轮工人2:



```
Do {  
    加工一个车轮;  
    SP(wm,1,1; empty,1,1);  
    车轮放入箱中;  
    SV(w,1);  
} while (True)
```

组装工人3:



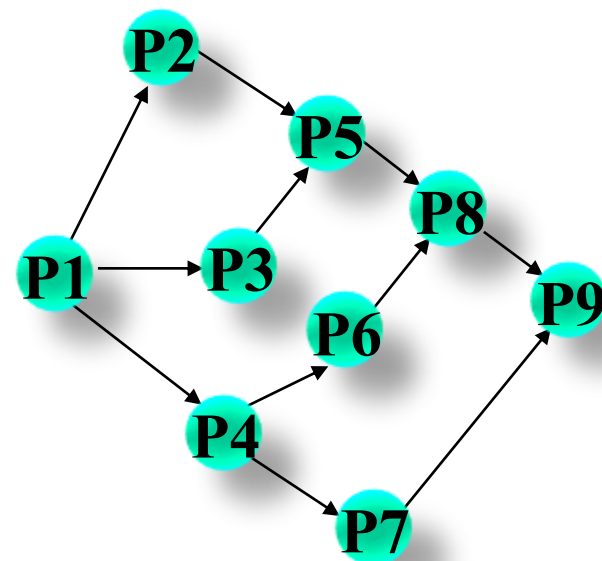
```
Do {  
    SP(f,1,1; w,2,2);  
    取1个车架2个车轮;  
    SV(empty,3; fm,1; wm,2);  
    加工一辆自行车;  
} while (True)
```



4.6 利用信号量实现前趋关系

4.6.1 前趋图定义

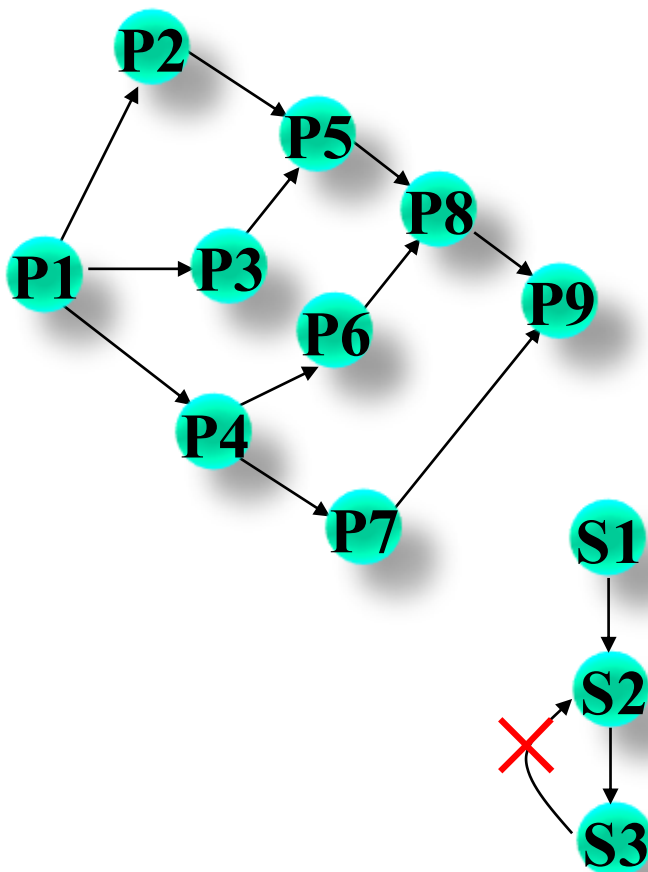
- 前趋图是一个有向无循环图，用于描述进程之间执行的前后关系；
- 每个结点表示一个进程、程序段或一条语句；结点间的有向边表示两结点间存在的偏序或前趋关系；
- $P_i \rightarrow P_j$: P_i 是 P_j 的直接前趋， P_j 是 P_i 的直接后继，表示在 P_j 可能开始执行前 P_i 必须执行完成；
- 给图写出所有的前趋关系。





4.6 利用信号量实现前趋关系

4.6.1 前趋图定义



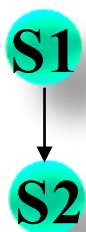
$P1 \rightarrow P2, P1 \rightarrow P3, P1 \rightarrow P4,$
 $P2 \rightarrow P5, P3 \rightarrow P5, P4 \rightarrow P6,$
 $P4 \rightarrow P7, P5 \rightarrow P8, P6 \rightarrow P8,$
 $P7 \rightarrow P9, P8 \rightarrow P9$

前趋图中不能存在循环



4.6 利用信号量实现前趋关系

4.6.2 前趋关系描述



进程P1: ... S1 ...

进程P2: ... S2 ...

若希望在S1执行后再执行S2，只需使
进程P1和P2共享一个私有信号量S，
并赋初值0

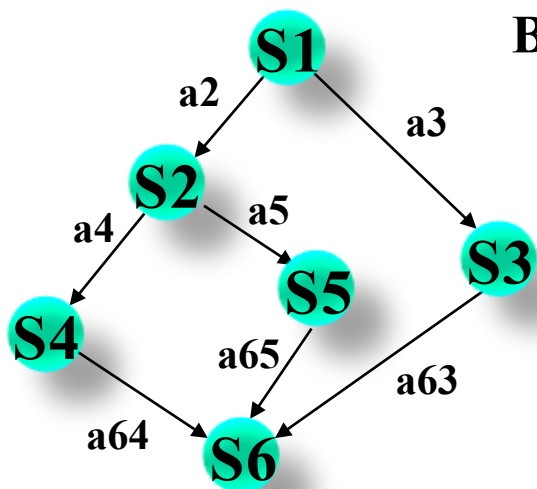
进程P1: ... S1; **V(S)**; ...

进程P2: ... **P(S)**; S2 ...



4.6 利用信号量实现前趋关系

4.6.2 前趋关系描述



Var

a2,a3,a4,a5,a63,a64,a65 :=semaphore:=0,0,0,0,0,0,0;

Begin

Parbegin

Begin S1;V(a2);V(a3); End

Begin P(a2);S2;V(a4);V(a5); End

Begin P(a3);S3;V(a63); End

Begin P(a4);S4;V(a64); End

Begin P(a5);S5;V(a65); End

Begin P(a63); P(a64); P(a65); S6; End

Parend

End

总结：前驱图中的边指明了消息的来源和去处，**可以对应一个信号量**



进程同步问题小结



- 通过分析前述经典的进程同步问题的解决方法，可以总结出进程同步的编程实现方法：
 - 首先，**编写程序核心代码**；
 - 然后，**分析进程同步关系**；即分析各进程或其中的程序段或语句的**执行条件**，设置相应的信号量
 - 最后，分析进程互斥关系。即分析哪些变量或资源需要互斥修改

进程同步分析方法：

- ①找出需要同步的代码片段（关键代码），建议画图展示同步关系；
- ②分析所找代码片段的执行次序；
- ③增加同步信号量S并赋初值；
- ④所找代码片段前后加P(S)或wait(S)和V(S)或signal(S)操作。

进程互斥分析方法：

- ①查找临界资源；
- ②划分临界区；
- ③定义互斥信号量S并赋初值；
- ④在临界区前后的进入区和退出区中分别加入wait(S)和signal(S)操作。

引申：合作者间遵守协议的重要性->契约精神->守信



4.7 管程(monitor)



4.7.1 信号量同步的缺点

- ① **同步操作分散**：信号量机制中，同步操作分散在各个进程中，使用不当就可能导致各进程死锁（如P、V操作的次序错误、重复或遗漏）；
- ② **易读性差**：要了解对于一组共享变量及信号量的操作是否正确，必须通读整个系统或者并发程序；
- ③ **不利于修改和维护**：各模块的独立性差，任一组变量或一段代码的修改都可能影响全局；
- ④ **正确性难以保证**：操作系统或并发程序通常很大，很难保证这样一个复杂的系统没有逻辑错误。

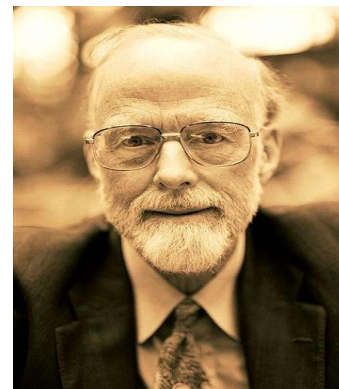
想想： 这个很像什么技术解决的编程中的问题？



4.7 管程(monitor)

4.7.2 管程的定义

- 用信号量可实现进程间的同步，但由于信号量的控制分布在整个程序中，其正确性分析很困难。易发生死锁。
- 1973年，Hoare和Hanson所提出；其基本思想是把信号量及其操作原语封装在一个对象内部。即：将共享变量以及对共享变量能够进行的所有操作集中在一个模块中。
 - Hoare, 1980年图灵奖得主，托尼·霍尔的“计算机程序设计的公理基础”是程序设计理论中最有影响力的论文之一





4.7 管程(monitor)

4.7.2 管程的定义

- **管程的定义**：管程是关于共享资源的数据结构及一组针对该资源的操作过程所构成的软件模块。
- **管程**是管理进程间同步的机制，它保证进程互斥地访问共享变量，并方便地阻塞和唤醒进程。
- 管程可以**函数库**的形式实现。相比之下，管程比信号量好控制。
- **管程**是一个程序设计语言结构，提供了与信号量同样的功能，但更易于控制。在并发Pascal、Modula2、**Java**中得以实现。



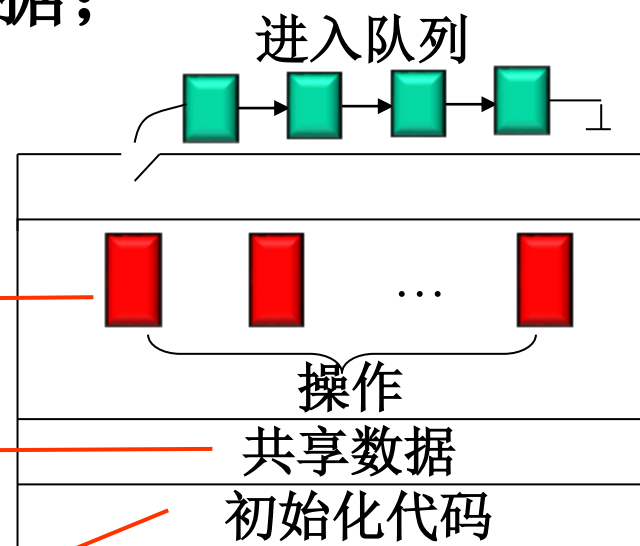
4.7 管程(monitor)

4.7.2 管程的定义

● 一个管程定义了一个**数据结构**和能为并发进程所执行（在该数据结构上）的**一组操作**，这组操作能同步进程和改变管程中的数据；

● 管程的组成：

- ✓ 管程的**名称**
- ✓ 对该数据结构进行操作的一**组过程**
- ✓ 局部于管程内部的**共享数据结构说明**
- ✓ 对局部于管程内部的共享数据设置**初始值的语句**





4.7 管程(monitor)

4.7.2 管程的定义

TYPE monitor_name = MONITOR;

共享变量说明

define 本管程内所定义、本管程外可调用的过程（函数）名字表

use 本管程外所定义、本管程内将调用的过程（函数）名字表

PROCEDURE 过程名（形参表）；

过程局部变量说明；

BEGIN 语句序列； **END**；

.....

FUNCTION 函数名（形参表）： 值类型；

函数局部变量说明；

BEGIN 语句序列； **END**；

.....

BEGIN 共享变量初始化语句序列； **END**；



4.7 管程(monitor)

4.7.2 管程的定义

● 管程的特点:

- 其内部的局部数据变量只能被管程内定义的过程所访问，不能为管程外声明的过程直接访问
- 进程若想进入管程，必须调用管程内的某个过程
- 一次只能有一个进程在管程内执行，其余调用该管程的过程均被阻塞，等待该管程成为可用的
- 管程能有效地实现互斥

类似于面向对象软件中的类



4.7 管程(monitor)

4.7.3 管程的主要特征

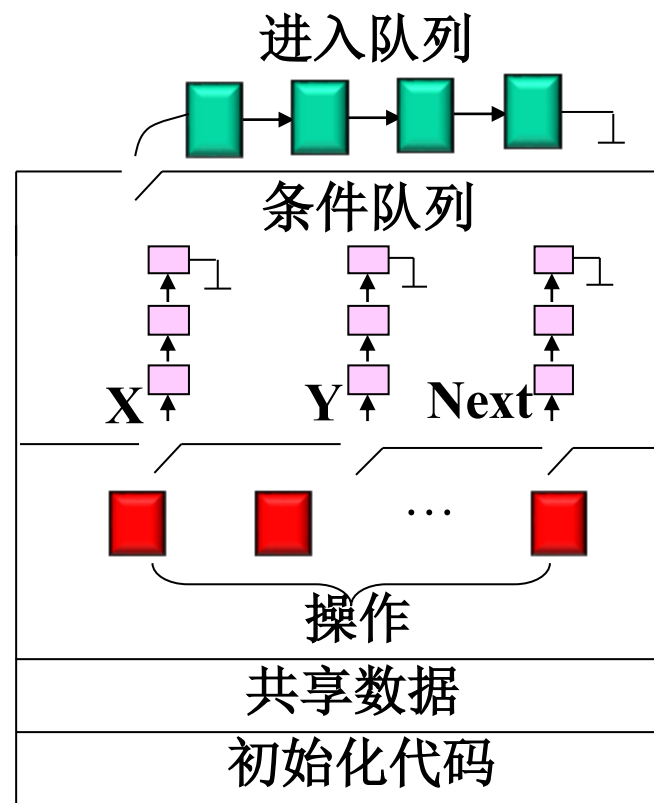
- **模块化**：一个管程是一个基本程序单位，可以单独编译；
- **抽象数据类型**：管程是一种特殊的数据类型，其中不仅有数据，而且有对数据进行操作的代码
- **信息掩藏**：管程是半透明的，管程中的外部过程（函数）实现了某些功能，至于这些功能是怎样实现的，在其外部则是不可见的；



4.7 管程(monitor)

4.7.4 条件变量

- 为解决同步问题，引入**条件变量**及两个相关操作原语：**wait**和**signal**。条件变量包含在管程内，且只能在管程内对它进行访问；
- Wait(x)**：挂起等待条件x的调用进程，释放相应的管程，以便其它进程使用；
- Signal(x)**：恢复执行先前因在条件x上执行wait而挂起的那个进程。





4.7 管程(monitor)



4.7.4 条件变量

● **Signal(x)**: 恢复执行先前因在条件x上执行wait而挂起的那个进程。

- ✓ 如果存在多个这样的进程，则选择其中一个；如果没有，则继续执行原进程，不产生任何结果——与信号量的V操作不同，V操作总要执行+1操作
- ✓ 如果有进程Q因条件x被阻塞，当正在调用管程的进程P执行了Signal(x)后，进程Q被重新启动，此时两个进程P和Q，哪个执行，哪个等待，可采取两种方式
 - ① P等待，直到执行Q离开管程或下一次等待。
 - ② Q送入Ready队列，直到执行P离开管程或下一次等待



4.7 管程(monitor)

4.7.5 管程的例子：读者、写者问题

TYPE r_and_w=MONITOR

VAR instance : one_instance

ra.wa : semaphore;

管程内定义，
外部调用

w_count : integer;

read_count, write_count

define start_read, finish_read

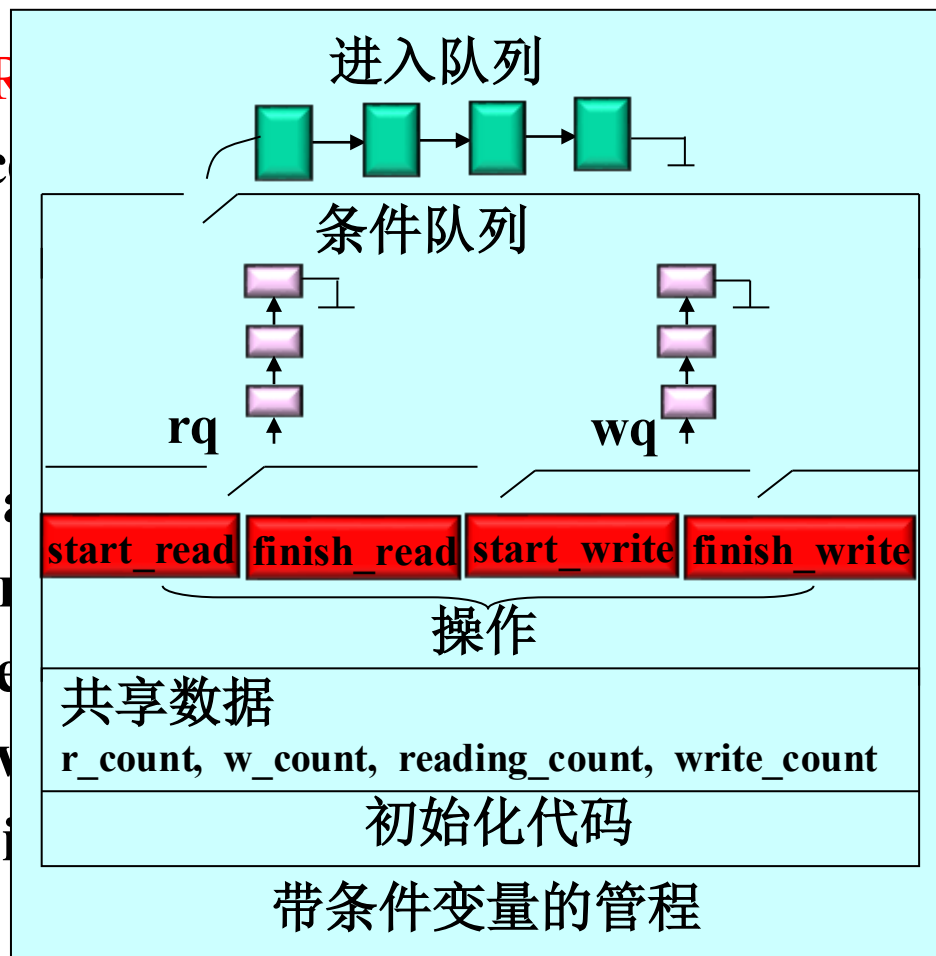
use monitor_elements.e

monitor_elements.l

monitor_elements.w

monitor_elements.s

管程外定义，
内部调用

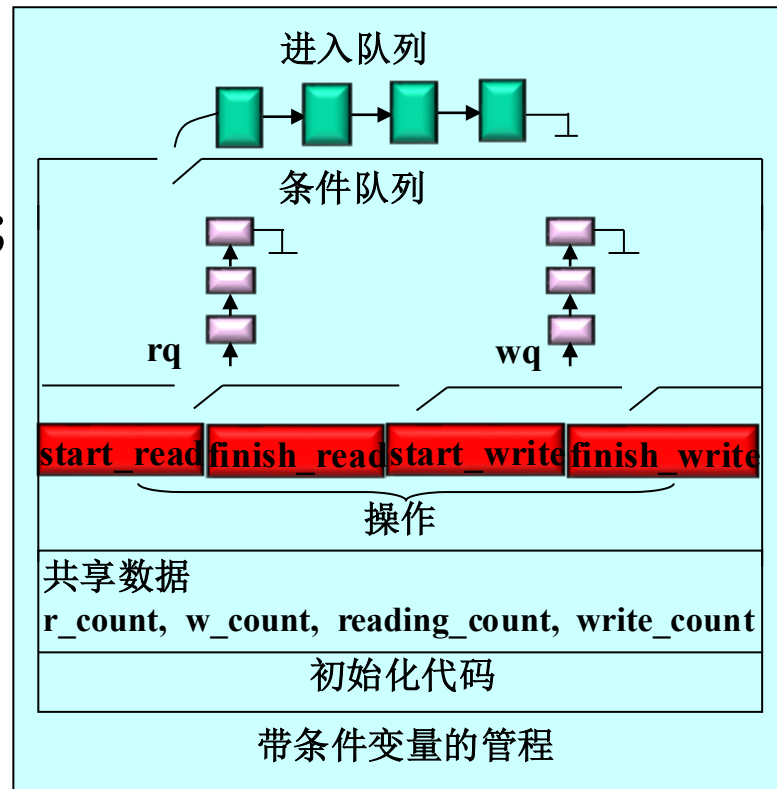




4.7 管程(monitor)

4.7.5 管程的例子：读者、写者问题

```
PROCEDURE start_read;  
BEGIN  
    monitor_elements.enter(instance);  
    IF write_count > 0 THEN  
        monitor_elements.wait(instance,  
                                rq, r_count);  
    reading_count++;  
    monitor_elements.leave(instance);  
    // 保证管程中只有一个进程  
END;
```

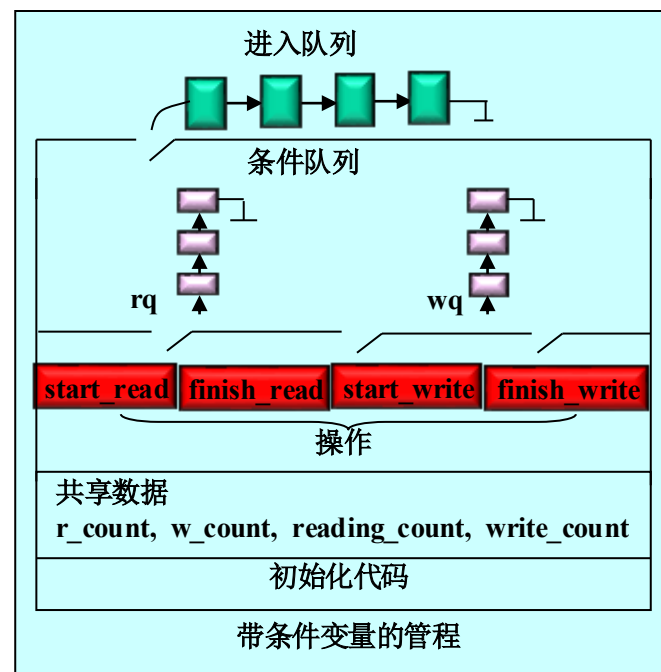




4.7 管程(monitor)

4.7.5 管程的例子：读者、写者问题

```
PROCEDURE finish_read;  
BEGIN  
    monitor_elements.enter(instance);  
    reading_count--;  
    IF reading_count=0 THEN  
        monitor_elements.signal(instance,  
                                     wq,w_count);  
    monitor_elements.leave(instance);  
END;
```





4.7 管程(monitor)

4.7.4管程的例子：读者、写者问题

```
PROCEDURE start_write;  
BEGIN
```

```
    monitor_elements.enter(instance);
```

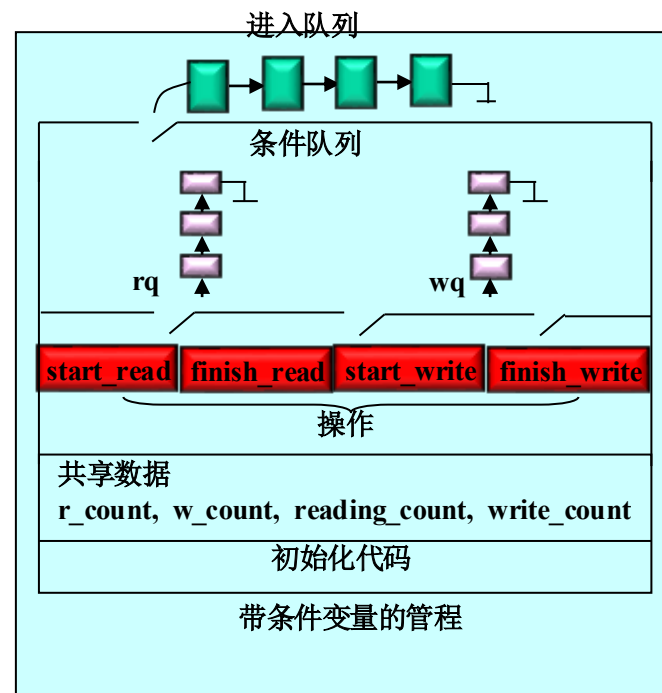
```
    write_count++;
```

```
    IF (write_count>1) OR (reading_count>0)
```

```
    THEN monitor_elements.wait(instance,  
                                wq,w_count);
```

```
    monitor_elements.leave(instance);
```

```
END;
```

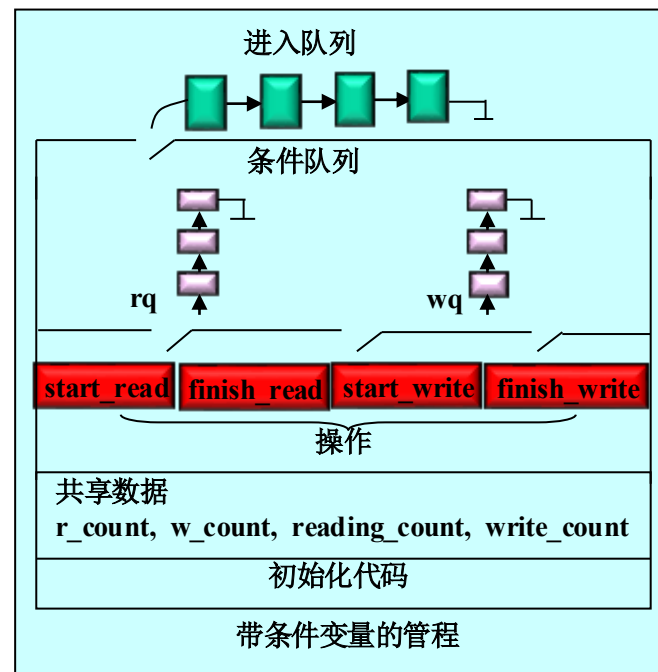




4.7 管程(monitor)

4.7.4管程的例子：读者、写者问题

```
PROCEDURE finish_write; // 写之后
BEGIN
    monitor_elements.enter(instance);
    write_count--;
    IF write_count > 0 THEN
        monitor_elements.signal(instance,
                                   wq, w_count);
    ELSE
        monitor_elements.signal(instance,
                                   rq, r_count);
    monitor_elements.leave(instance);
END;
```



```
PROCEDURE start_read;  
BEGIN  
    monitor_elements.enter(instance);  
    IF write_count>0 THEN  
        monitor_elements.-  
            wait(instance,rq,r_count);  
    reading_count++;  
    monitor_elements.leave(instance);  
END;
```

```
PROCEDURE start_write;  
BEGIN  
    monitor_elements.enter(instance);  
    write_count++;  
    IF (write_count>1) OR (reading_count>0)  
    THEN monitor_elements.-  
        wait(instance,wq,w_count);  
    monitor_elements.leave(instance);  
END;
```

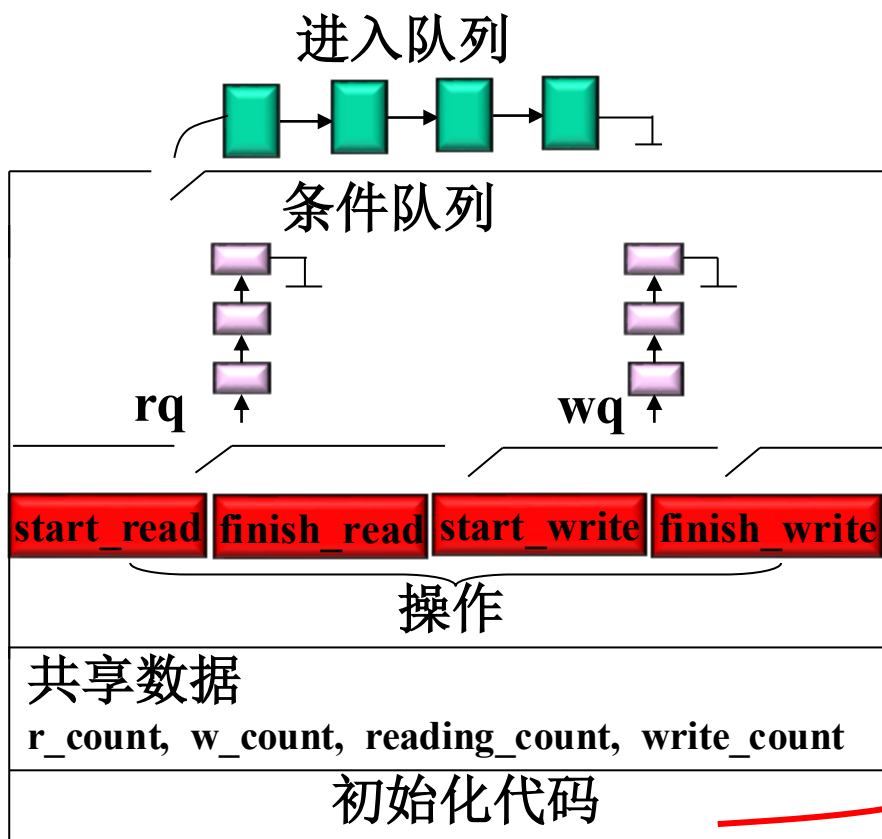
```
PROCEDURE finish_read;  
BEGIN  
    monitor_elements.enter(instance);  
    reading_count--;  
    IF reading_count=0 THEN  
        monitor_elements.-  
            signal(instance,wq,w_count);  
    monitor_elements.leave(instance);  
END;
```

```
PROCEDURE finish_write;  
BEGIN  
    monitor_elements.enter(instance);  
    write_count--;  
    IF write_count>0 THEN  
        monitor_elements.signal(instance,wq,w_count);  
    ELSE  
        monitor_elements.signal(instance,rq,r_count);  
    monitor_elements.leave(instance);  
END;
```



4.7 管程(monitor)

4.7.5管程的例子：读者、写者问题



```
BEGIN // 初始化部分
    reading_count:=0;
    write_count:=0;
    r_count:=0;
    w_count:=0;
END;
```

带条件变量的管程



4.7 管程(monitor)

4.7.5管程的例子：读者、写者问题

读者进程的活动：

.....

`r_and_w.start_read;`

读操作;

`r_and_w.finish_read;`

.....



写者进程的活动：

.....

`r_and_w.start_write;`

写操作;

`r_and_w.finish_write;`

.....



后跳



4.7 管程(monitor)

4.7.5管程的例子：生产者、消费者问题

Type producer-consumer=monitor

var in,out,count:integer;

buffer:array[0,...,n-1] of item;

notfull,notempty:condition;

procedure entry put(item)

begin

if count $\geq$ n then wait(notfull);

buffer(in)=item;

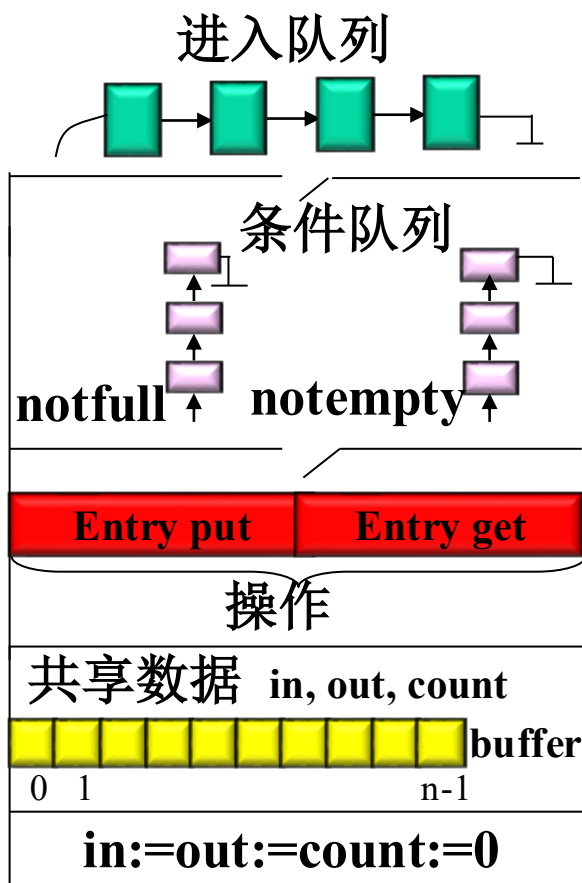
in=(in+1) mod n

count=count+1;

if notempty.queue then

signal(notempty);

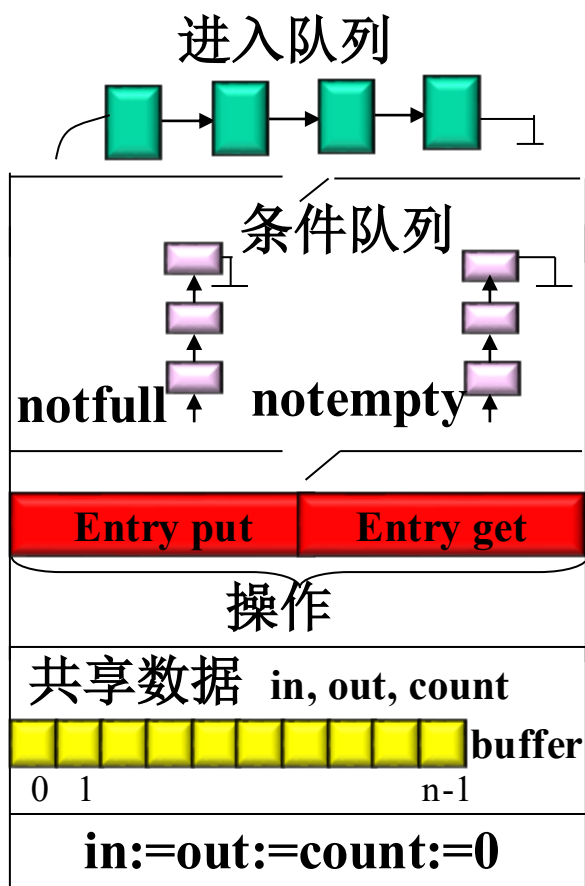
end





4.7 管程(monitor)

4.7.5管程的例子：生产者、消费者问题

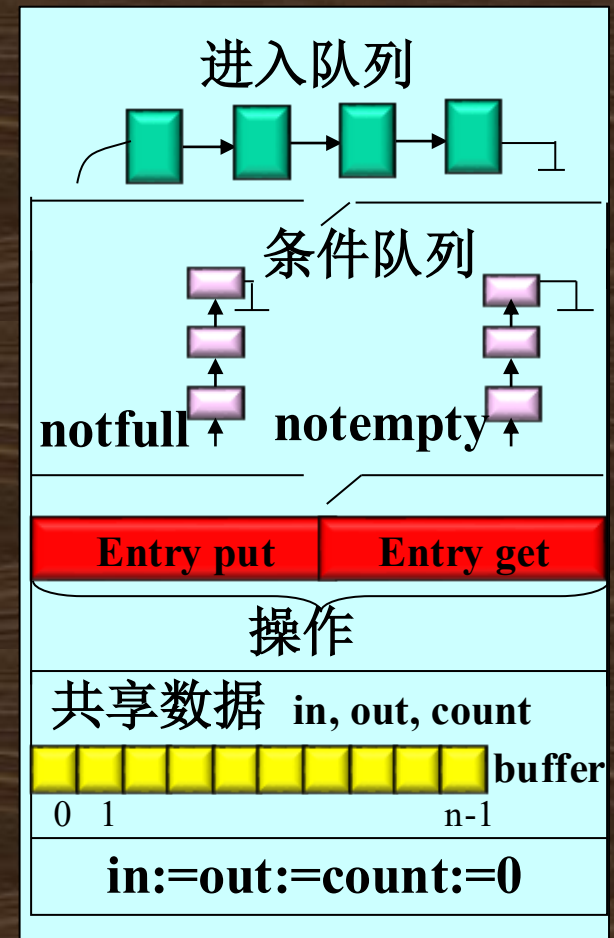


```
procedure entry get(item)
begin
  if count<=0 then wait(notempty);
  item:=buffer(out);
  out:=(out+1) mod n;
  count:=count-1;
  if notfull.queue then signal(notfull);
end
begin
  in:=out:=0;
  count:=0
end
```

Type producer-consumer=monitor

```
var in,out,count:integer;  
    buffer:array[0,...,n-1] of item;  
    notfull,notempty:condition;  
    procedure entry put(item)  
    begin  
        if count>=n then wait(notfull);  
        buffer(in)=item;  
        in=(in+1) mod n  
        count=count+1;  
        if notempty.queue then signal(notempty);  
    end
```

```
    procedure entry get(item)  
    begin  
        if count<=0 then wait(notempty);  
        item:=buffer(out);  
        out:=(out+1) mod n;  
        count:=count-1;  
        if notfull.queue then signal(notfull);  
    end
```





4.7 管程(monitor)

4.7.5管程的例子：生产者、消费者问题

生产者进程：

begin

repeat

produce an item;

PC.put(item);

until false;

end



消费者进程：

begin

repeat

PC.get(item);

consume the item;

until false;

end





4.7 管程(monitor)

读者-写者、生产者-消费者问题比较



读者进程的活动:

```
.....  
r_and_w.start_read;  
读操作;  
r_and_w.finish_read;  
.....
```



写者进程的活动:

```
.....  
r_and_w.start_write;  
写操作;  
r_and_w.finish_write;  
.....
```



生产者进程:

```
begin  
  repeat  
    produce an item;  
    PC.put(item);  
  until false;  
end
```



消费者进程:

```
begin  
  repeat  
    PC.get(item);  
    consume the item;  
  until false;  
end
```



4.7 管程(monitor)

4.7.6 管程和进程的异同点

- 设置进程和管程的目的不同
- 系统管理数据结构
 - 进程: PCB
 - 管程: 等待队列
- 管程被进程调用
- 管程是操作系统的固有成分, 无创建和撤消
- 进程之间能并发执行, 而管程不能与其调用并发



4.7 管程(monitor)

4.7.7 管程的实现要素

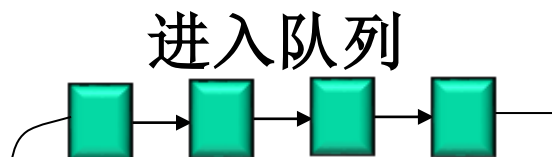
- 管程中的共享变量在管程外部是不可见的，外部只能通过调用管程中所说明的**外部过程**（函数）来间接地访问管程中的共享变量；
- 为了保证管程共享变量的数据完整性，规定**管程互斥进入**；
- 管程通常是用来管理资源的，因而在管程中应当设有**进程等待队列**以及相应的**等待及唤醒操作**。



4.7 管程(monitor)

4.7.7 管程的实现要素—管程的结构

- **入口等待队列：**因为管程是互斥进入的，所以当
一个进程试图进入一个已被占用的管程时它应当
在管程的入口处等待，因而在管程的入口处应当
有一个进程等待队列，称作入口等待队列。





4.7 管程(monitor)

4.7.7 管程的实现要素—管程的结构

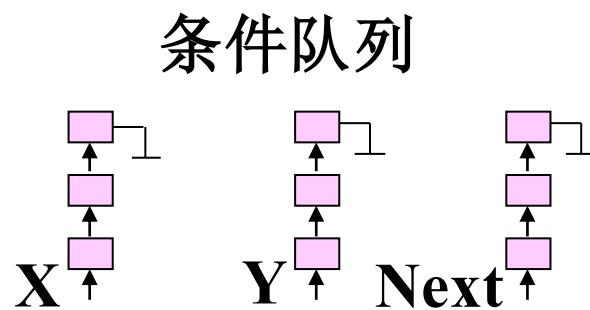
- **紧急等待队列：**如果进程P唤醒进程Q，则P等待Q继续，如果进程Q在执行又唤醒进程R，则Q等待R继续，...，如此，在管程内部，由于执行唤醒操作，可能会出现多个等待进程（**已被唤醒，但由于管程的互斥进入而等待**），因而还需要有一个进程等待队列，这个等待队列被称为紧急等待队列。它的优先级应当高于入口等待队列的优先级。



4.7 管程(monitor)

4.7.7 管程的实现要素—条件变量(condition)

- 每个条件变量表示一种等待原因，并不取具体数值——相当于每个原因对应一个队列。条件变量实际上是一个指针，它可以指向空，或者指向由进程控制块所构成队列的头部，初始时它指向空 (X、Y、Next)。
- 对于条件型变量，可以执行wait和signal操作。wait(x)将自己阻塞在x队列中，signal(x)将x队列中的一个进程唤醒。





4.7 管程(monitor)

4.7.7 管程的实现要素--条件变量(condition)

● 条件变量的wait (x)

wait(x)将自己阻塞在x队列中

- 如果紧急等待队列非空，则唤醒第一个等待者；
- 否则释放管程的互斥权，执行此操作的进程排入x队列尾部；
- 紧急等待队列与x队列的关系：
 - 紧急等待队列是由于管程的互斥进入而等待的队列；
 - x队列是因资源被占用而等待的队列。



4.7 管程(monitor)

4.7.7 管程的实现要素—条件变量(condition)

● **signal (x)**

signal(x)将x队列中的一个进程唤醒

- 如果x队列为空，则相当于空操作，执行此操作的进程继续；
- 否则唤醒第一个等待者，执行此操作的进程排入紧急等待队列的尾部；
- 若进程P唤醒进程Q，则随后可有两种执行方式（进程P、Q都是管程中的进程）：
 - P等待，直到执行Q离开管程或下一次等待。**Hoare**采用；
 - Q送入Ready队列，直到执行P离开管程或下一次等待。**1980年，Lampson和Redell**采用。原语由**csignal(x)**改为**cnotify(x)**。



4.7 管程(monitor)

4.7.7 管程的实现要素—管程的互斥

- 根据管程互斥的要求，进程进入管程和离开管程时，分别应当执行如下的操作：
 - 进入管程：申请管程的互斥权；
 - 离开管程：如果紧急队列非空，则唤醒第一个等待者，否则释放管程的互斥权。

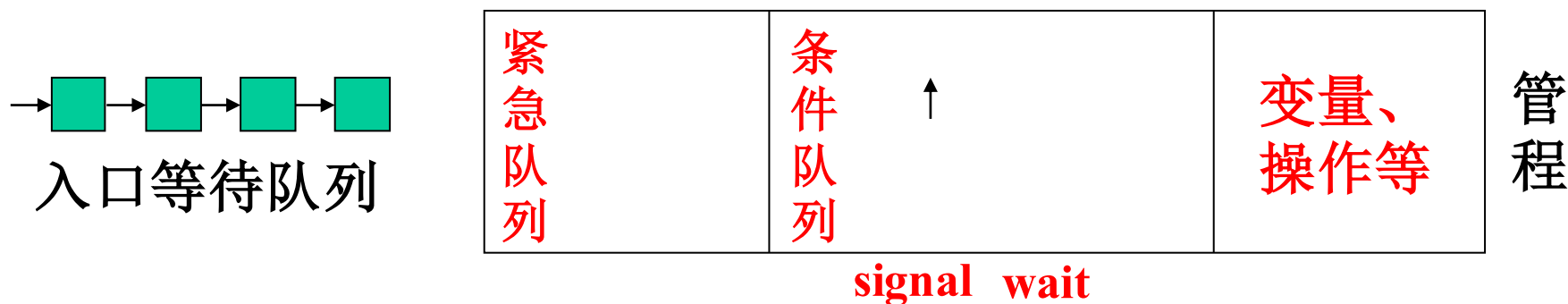


4.7 管程(monitor)

4.7.7 管程的实现要素—管程的互斥

● 根据管程互斥的要求，进程进入管程和离开管程时，分别应当执行如下的操作：

- 进入管程：申请管程的互斥权；
- 离开管程：如果紧急队列非空，则唤醒第一个等待者，否则释放管程的互斥权。



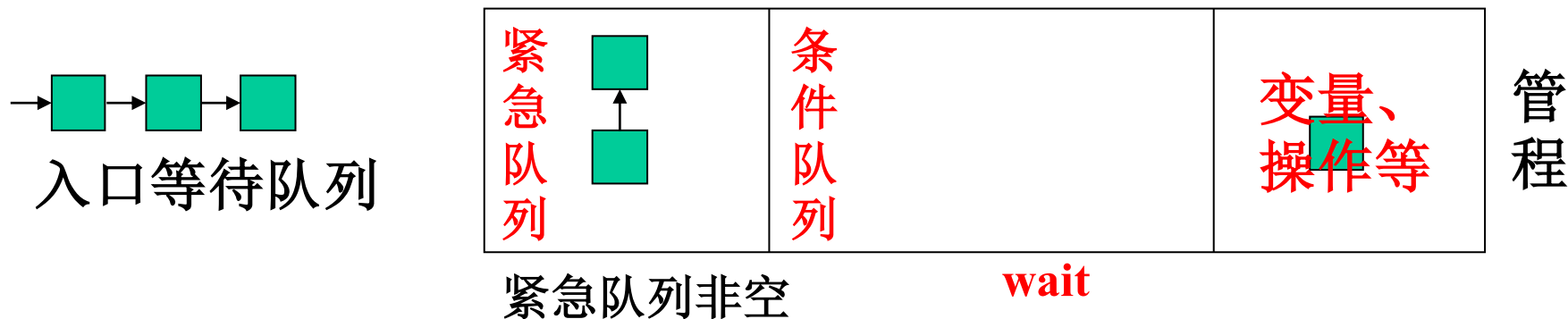


4.7 管程(monitor)

4.7.7 管程的实现要素—管程的互斥

● 根据管程互斥的要求，进程进入管程和离开管程时，分别应当执行如下的操作：

- 进入管程：申请管程的互斥权；
- 离开管程：如果紧急队列非空，则唤醒第一个等待者，否则释放管程的互斥权。





4.7 管程(monitor)

4.7.7 管程的实现要素—管程的实现

TYPE one_instance=RECORD

mutex : semaphore; (入口互斥队列, 初值1)

urgent : semaphore; (紧急等待队列, 初值0)

urgent_count : integer; (紧急等待计数, 初值0)

END;

TYPE

monitor_elements=MODULE;

define enter, leave, wait, signal;



4.7 管程(monitor)

4.7.7 管程的实现要素—管程的互斥

```
PROCEDURE enter(VAR instance:one_instance);  
    // 进入管程时执行该操作，申请管程的互斥权  
BEGIN  
    P(instance.mutex)  
END;
```



4.7 管程(monitor)

4.7.7 管程的实现要素—管程的互斥

```
PROCEDURE leave(VAR instance:one_instance);  
// 离开管程时执行， 唤醒紧急进程或释放互斥权  
BEGIN  
  IF instance.urgent_count > 0 THEN  
    BEGIN instance.urgent_count - -;  
          V(instance.urgent)  
    END  
  ELSE  
    V(instance.mutex)  
  END;
```



4.7 管程(monitor)

4.7.7 管程的实现要素—管程的实现

```
PROCEDURE wait(VAR instance : one_instance;  
    VAR s : semaphore;    //条件变量  
    VAR count : integer); // 在条件变量队列中等待的进程数目  
BEGIN  
    count++;                //等待的进程数目加1  
    IF instance.urgent_count>0 THEN  
        BEGIN  
            instance.urgent_count--; //如果紧急等待队列非空,  
            V(instance.urgent);      //则唤醒第一个等待者;  
        END  
    ELSE    V(instance.mutex);    //否则释放管程的互斥权,  
    P(s);    //执行此操作的进程排入s队列尾部  
END;
```




4.7 管程(monitor)

4.7.7 管程的实现要素—管程的实现

```
PROCEDURE signal(VAR instance:one_instance;  
    VAR s:semaphore;      //条件变量  
    VAR count:integer);    // 在条件变量队列中等待的进程数目  
BEGIN  
    IF count>0 THEN  
        BEGIN  
            count- -;  
            instance.urgent_count++;  
            V(s);          // s队列非空, 唤醒第一个等待者  
            P(instance.urgent) //执行此操作的进程排入紧急等待队列的尾部  
        END  
    END;  
    //如果s队列为空, 则相当于空操作, 执行此操作的进程继续;
```



4.7 管程(monitor)

4.7.7 管程的实现要素—管程的优点

- **管程可增强模块的独立性：**系统按资源管理的观点分解成若干模块，用数据表示抽象系统资源，同时分析了共享资源和专用资源在管理上的差别，按不同的管理方式定义模块的类型和结构，使同步操作相对集中，从而增加了模块的相对独立性；
- **引入管程可提高代码的可读性，**便于修改和维护，正确性易于保证：采用集中式同步机制。一个操作系统或并发程序由若干个这样的模块所构成，一个模块通常较短，模块之间关系清晰。



4.8 Linux的进程同步机制【自学】

● Linux系统包含多种的同步机制：

- ① 自旋锁（spinlock）
- ② 信号量（semaphone）
- ③ 原子操作（atomic operation）
- ④ 读写（rwlock）
- ⑤ RCU（Read-copy update）
- ⑥ seqlock

● 每种机制应用在不同的场合，这些机制的发展伴随Linux从单处理器到对称多处理器的过渡，从非抢占式内核到抢占式内核的过渡，锁机制越来越有效，也越来越复杂。



4.8 Linux的进程同步机制【自学】

4.8.1 自旋锁

- 原理：自旋的意思就是一直循环直到条件满足。自旋锁不会引起调用者的睡眠，如果自旋锁已经被别的执行单元占有，调用者就一直循环查看是否该自旋锁的保持者已经释放了锁。如果不能在很短的时间内获得锁，则无疑会导致CPU效率降低。
- 相关的函数主要包括如下几个：
 - ① `spin_lock_init(spinlock_t*)` //初始化自旋锁
 - ② `spin_lock(spinlock_t*)` //循环等待获取自旋锁
 - ③ `spin_trylock(spinlock_t*)` //非阻塞方式
 - ④ `spin_unlock(spinlock_t*)`
 - ⑤ `spin_lock_irq()`
 - ⑥ `spin_lock_irqsave()`
 - ⑦ `spin_lock_bh(spinlock_t*)`

```
#include <linux/rtmutex.h>
typedef struct spinlock {
    struct rt_mutex_base lock;
#ifdef CONFIG_DEBUG_LOCK_ALLOC
    struct lockdep_map dep_map;
#endif
} spinlock_t;
```



4.8 Linux的进程同步机制【自学】

4.8.2 信号量

Linux提供两种信号量：

- ① 内核信号量，由内核控制路径使用
- ② 用户态进程使用的信号量，这种信号量又分为POSIX信号量和SYSTEM V信号量。
- ③ POSIX信号量又分为有名信号量 and 无名信号量。
 - ✓ 有名信号量，其值保存在文件中，所以它可以用于线程也可以用于进程间的同步；
 - ✓ 无名信号量，其值保存在内存中。



4.8 Linux的进程同步机制【自学】

4.8.2 信号量--Linux内核信号量:

- ① 内核信号量类似于自旋锁，因为当锁关闭着时，它不允许内核控制路径继续进行。然而，当内核控制路径试图获取内核信号量锁保护的忙资源时，相应的进程就被挂起。只有在资源被释放时，进程才再次变为可运行。
- ② 只有**可以睡眠的函数才能获取内核信号量**；中断处理程序和可延迟函数都不能使用内核信号量。



4.8 Linux的进程同步机制【自学】

4.8.2 信号量--Linux内核信号量:

- ✓ 内核信号量是struct semaphore类型的对象，它在<asm/semaphore.h>中定义：

```
struct semaphore {  
    undefined atomic_t count;  
    int sleepers;  
    wait_queue_head_t wait;  
}
```

- ✓ **count**: 相当于信号量的值，大于0，资源空闲；等于0，资源忙，但没有进程等待这个保护的资源；小于0，资源不可用，并至少有一个进程等待资源。
- ✓ **wait**: 存放等待队列链表的地址，当前等待资源的所有睡眠进程都会放在这个链表中。
- ✓ **sleepers**: 存放一个标志，表示是否有一些进程在信号量上睡眠。



4.8 Linux的进程同步机制【自学】



4.8.2信号量--Linux内核信号量:

内核信号量的相关函数

- 初始化:

- `void sema_init (struct semaphore *sem, int val);`
- `void init_MUTEX (struct semaphore *sem);` //将sem的值置为1
- `void init_MUTEX_LOCKED (struct semaphore *sem);` //将sem的值置为0, 表示资源忙

- 申请内核信号量所保护的资源:

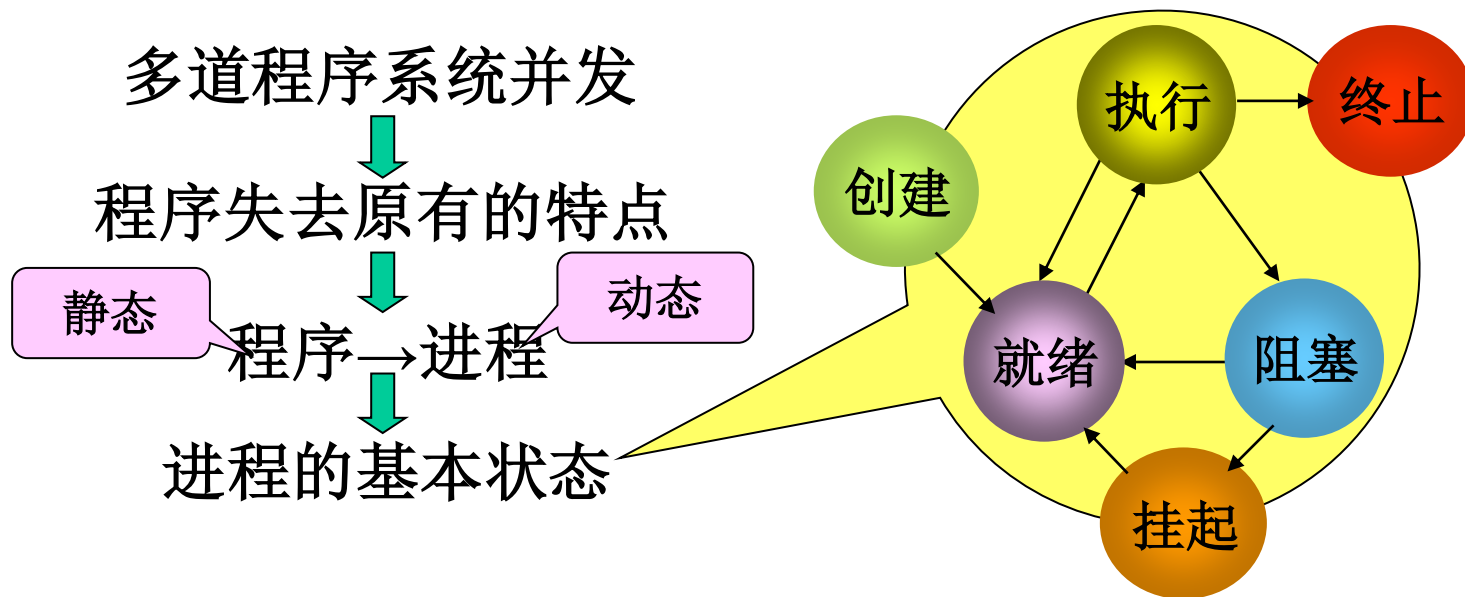
- `void down(struct semaphore * sem);` // 可引起睡眠
- `int down_interruptible(struct semaphore * sem);` // 能被信号打断
- `int down_trylock(struct semaphore * sem);` // 非阻塞函数, 不会睡眠。无法锁定资源则马上返回

- 释放内核信号量所保护的资源:

- `void up(struct semaphore * sem);`

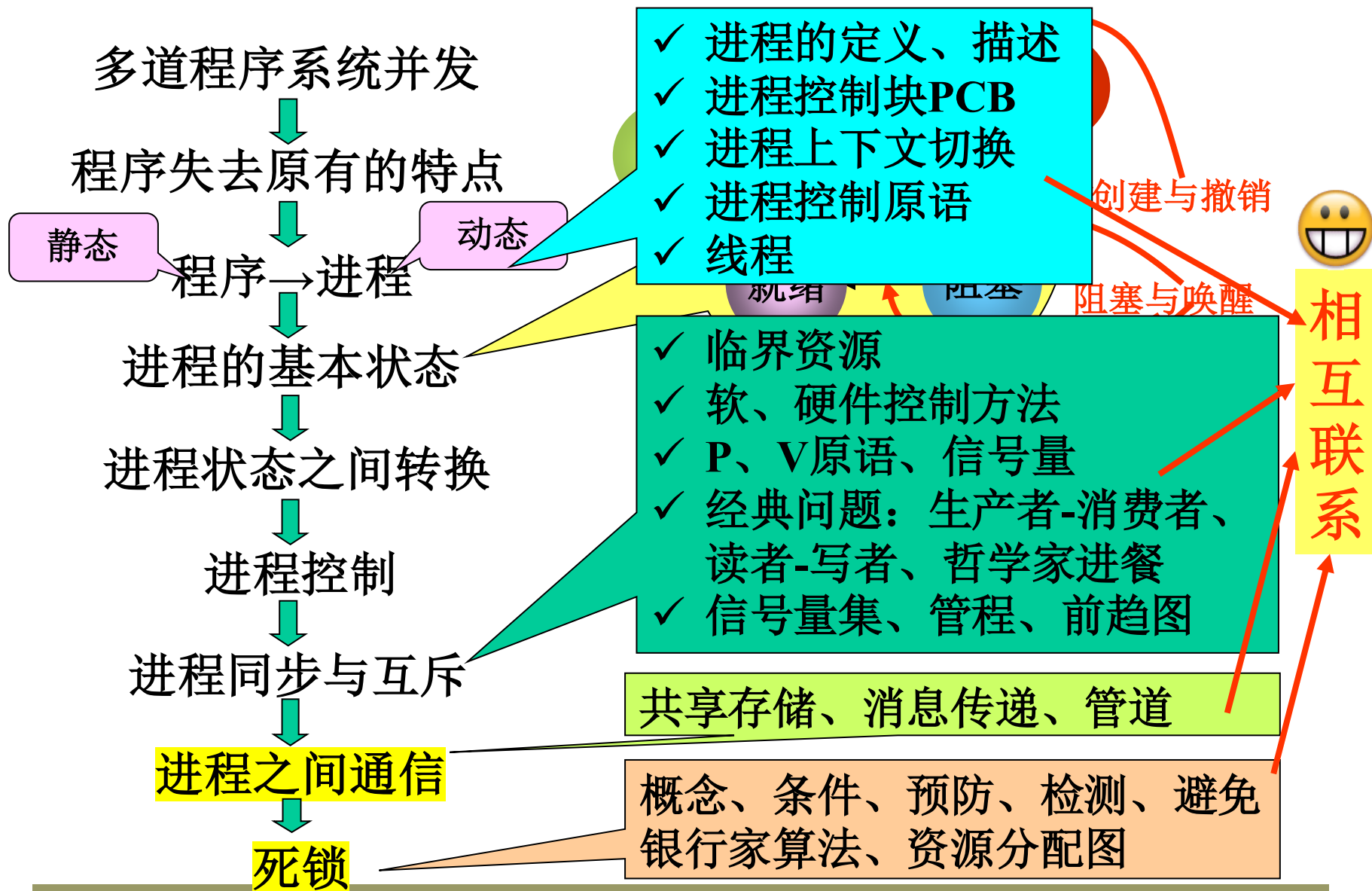


进程管理小结





第四章小结





作业



1. 今有三个并发进程R, M, P, 它们共享了一个可循环使用的缓冲区B, 缓冲区B共有N个单元。进程R负责从输入设备读信息, 每读一个字符后, 把它存放在缓冲区B的一个单元中; 进程M负责处理读入的字符, 若发现读入的字符中有空格符, 则把它改成“,”; 进程P负责把处理后的字符取出并打印输出。当缓冲区单元中的字符被进程P取出后, 则又可用来存放下一次读入的字符。请用PV操作作为同步机制写出它们能正确并发执行的程序。
2. 有一个仓库可以存放A、B两种物品, 每次只能存入一件物品 (A或B)。存储空间足够大, 只是要求 $-n < (A \text{的件数} - B \text{的件数}) < m$, 其中n和m是正整数。试用存入A、存入B和P、V操作, 描述物品A和物品B的入库过程。
3. 学习: 用管程描述生产者和消费者问题



继续学习第5章： 进程间通信

